# Shortest Path Algorithm with Heaps

—— 基于斐波那契堆、二项堆与二叉堆的 Dijkstra 实现与实验评估 ——

课程：高级数据结构与算法（ADS）· Project 2
题目：8-1 Shortest Path Algorithm with Heaps
姓名：【】　　学号：【】　　专业班级：【】
日期：2026 年 10 月

测试平台：13th Gen Intel Core i9-13900HX（32 逻辑核）/ 16 GB RAM / Windows 11 + MinGW-w64 GCC 14

## 摘要

本报告完成了题目 8-1 的全部要求：用 **二叉堆、二项堆和斐波那契堆** 三种最小优先队列分别实现 Dijkstra 最短路算法，在 DIMACS Challenge 9 的美国公路网基准上进行大规模实验（7 个 数据集、每个数据集 1000 组随机点对查询），并给出运行时间、堆操作次数与比较次数的完整统计。

报告第一部分（Chapter 2）重点介绍了 **斐波那契堆** 这一课外数据结构：它的结构、五种基本操作的伪代码，以及用势能法对 extract-min 的 $O(\log n)$ 和 decrease-key 的 $O(1)$ 摊还界的**完整证明**，并给出三种堆的复杂度对比。第二部分（Chapter 3、4）给出全部实现、正确性测试与性能测试结果，并分析了「理论最优 ≠ 实践最优」这一经典现象。

核心结论：三种堆在同一批查询上完成的**松弛次数完全相同**（差异 < 0.001%），因此耗时差异完全来自 堆本身。实测中 **斐波那契堆并没有获胜**——它比二叉堆慢 1.98 倍，二项堆慢 1.91 倍； 原因在于（i）公路网平均度很小、decrease-key 占比低，斐波那契堆摊还 $O(1)$ 的 decrease-key 优势 无从发挥；（ii）其 extract-min 需要 consolidate，常数因子大。而在最极端的「完整单源」实验中，三者的相对差距缩小到 1.5 倍以内，与理论预测的趋势一致。

# 目录


摘要 ..... 1

Chapter 1 Introduction ..... 3

1.1 问题描述 ..... 3

1.2 本报告的任务与目标 ..... 3

1.3 三种堆的背景 ..... 3

1.4 报告组织 ..... 3

Chapter 2 数据结构与算法规范 ..... 4

2.1 图的存储：CSR ..... 4

2.2 Dijkstra 算法框架 ..... 4

2.3 二叉堆（Binary Heap） ..... 5

2.4 二项堆（Binomial Heap） ..... 5

2.5 斐波那契堆（Fibonacci Heap） ..... 5

2.6 三种堆的复杂度对比 ..... 8

2.7 工程实现要点 ..... 8

Chapter 3 测试结果 ..... 9

3.1 正确性测试 ..... 9

3.2 实验环境 ..... 9

3.3 数据集 ..... 10

3.4 点对查询（1000 组随机点对） ..... 10

3.5 完整单源最短路（16 个随机源点） ..... 11

3.6 堆操作与比较次数统计 ..... 12

3.7 附加实验：耗时与搜索规模的关系 ..... 13

Chapter 4 分析与评论 ..... 15

4.1 实测结论汇总 ..... 15

4.2 为什么斐波那契堆在实践中没有获胜 ..... 15

4.3 空间复杂度 ..... 15

4.4 与其它数据结构 / 算法的比较 ..... 16

4.5 进一步可能的改进 ..... 16

参考文献 ..... 17

Appendix A 源代码 ..... 18

A.1 `src/heaps.h`（三种堆的实现，注释行占比 > 30%） ..... 18

A.2 `src/dijkstra.h`（Dijkstra 框架） ..... 28

A.3 `src/dimacs.h`（图与查询的读取） ..... 31

A.4 `src/bench.cpp`（性能测试程序） ..... 34

A.5 `src/selftest.cpp`（正确性测试） ..... 38

Appendix B 正确性测试完整输出 ..... 44

B.1 单元测试与随机对拍（`build/selftest.exe`） ..... 44

B.2 逐查询答案交叉验证（`tools/verify_answers.py`） ..... 44

作者分工与声明 ..... 46

作者分工 ..... 46

声明 ..... 46

# Chapter 1 Introduction

## 1.1 问题描述

最短路径问题是组合优化中最基础的问题之一。给定带非负权重的有向图 $G = (V, E)$ 以及源点 $s$， 单源最短路（SSSP）要求出 $s$ 到所有顶点的最短距离；若只关心某个终点 $t$，则称为点到点最短路（P2P）。

Dijkstra 算法是求解该问题的经典算法，其正确性与效率**强依赖于最小优先队列（min-priority queue）**的实现。在最坏情形下，一趟 Dijkstra 需要执行 $|V|$ 次 extract-min、至多 $|E|$ 次 decrease-key，因此优先队列的单次操作代价直接决定整个算法的复杂度：

$$O((n+m)\log n) \text{ （二叉堆 / 二项堆）} \quad \text{vs.} \quad O(m + n\log n) \text{ （斐波那契堆）}$$

题目要求在**至少两种**堆结构上实现 Dijkstra（其中必须包含斐波那契堆），并在 DIMACS Challenge 9 的美国公路网基准上评测，目标是「找出最适合 Dijkstra 的数据结构」。

## 1.2 本报告的任务与目标

| 题目要求 | 本报告的做法 |
|---|---|
| 至少两种堆、必须含斐波那契堆 | 实现二叉堆、二项堆、斐波那契堆三种，接口完全一致，另加一种“惰性删除”写法作对照 |
| 使用 USA 公路网评测（只列下载链接） | 使用 7 个网络，共 264,346 – 1,890,815 个结点；链接见表 3.1 |
| 至少 1000 组查询 | 每个数据集 1000 组随机点对，另在完整单源模式上再测 16 个源点 |
| 性能测试程序 | `bench`：自动遍历所有堆、输出 CSV 时间表与逐查询明细 |
| 运行时间表 + 规模曲线 | 表 3.2 – 3.5 与图 3.1 – 3.5 |
| 充分注释的代码 | Appendix A 全文收录，注释行占比 > 30% |

## 1.3 三种堆的背景

**二叉堆（binary heap）**是最常见的优先队列实现：用数组存完全二叉树，父结点不大于子结点。它缓存友好、常数极小，是工程实践中使用最广的选择；缺点是不支持高效的合并，且 decrease-key 需要 $O(\log n)$。

**二项堆（binomial heap，Vuillemin 1978）**是若干棵「二项树」$B_k$ 组成的森林，$B_k$ 有 $2^k$ 个 结点、度数为 $k$。二项堆第一次把 **合并（union）** 做到 $O(\log n)$，并让 insert 的摊还代价降为 $O(1)$；但 extract-min 与 decrease-key 的最坏代价仍是 $O(\log n)$。

**斐波那契堆（Fibonacci heap，Fredman & Tarjan 1987）** 进一步「偷懒」：insert、合并都只做 $O(1)$ 的挂牌操作，把所有整理工作推迟到 extract-min；decrease-key 则通过**切断（cut）** 直接完成， **摊还** 代价 $O(1)$。它使 Dijkstra 的复杂度从 $O(m\log n)$ 降到 $O(m + n\log n)$，是理论上的最优选择，也是后续许多图算法（如 $O(m\log n)$ 最小生成树、$O(n\log n + m)$ 单源最短路）的基石。斐波那契堆是本项目中**唯一的课外内容**，Chapter 2 将给出完整的原理与摊还分析证明。

## 1.4 报告组织

Chapter 2 给出图存储、Dijkstra 框架以及三种堆的规范与伪代码，并重点证明斐波那契堆的摊还复杂度；Chapter 3 给出正确性测试、数据集说明与全部性能测试结果；Chapter 4 分析数据、解释理论与实测的差异 并讨论改进方向；Appendix A 收录全部源代码。

# Chapter 2 数据结构与算法规范

## 2.1 图的存储：CSR

实验中最大 的图有 1,890,815 个结点、4,657,742 条弧，必须使用紧凑且缓存友好的存储。本项目采用 **CSR（Compressed Sparse Row）**，用三个数组表示整张图：

```
head[0 .. n]      // head[u]..head[u+1]-1 就是顶点 u 的出边区间
to[0 .. m-1]     // 弧头
w [0 .. m-1]     // 弧权
```

顶点 $u$ 的全部出边是 `to[head[u] .. head[u+1]-1]`，是一段连续内存，因此遍历出边时 几乎没有额外指针跳转。读取 `.gr` 文本文件时采用「两遍扫描」：第一遍只统计每个顶点的出度， 前缀和后得到 `head`；第二遍才填充 `to`/`w`。这样无需缓存 $(u, v, w)$ 三元组，内存占用最小， 190 万结点的图载入时间约 0.3 s（不计入算法计时）。

## 2.2 Dijkstra 算法框架

### 2.2.1 可索引堆 + decrease-key（标准写法）

三种堆都实现为「可索引」（indexed）优先队列，即堆内部维护 顶点 → 堆结点 的映射， 从而直接支持 decrease-key。这样写既符合教材中三种堆的标准形式，也保证了公平比较。

```
DIJKSTRA(G, w, s, t)                    // 可索引堆版本
  for each v in V: dist[v] ← +∞, settled[v] ← false
  dist[s] ← 0 ; H.INSERT(s, 0)

  while not H.EMPTY():
      u ← H.EXTRACT-MIN()                    // u 的最短距离已确定
      settled[u] ← true
      if u = t: return dist[u]               // 点到点查询：提前终止
      for each arc (u, v, w) with u → v:
          if settled[v]: continue
          if dist[u] + w < dist[v]:
              dist[v] ← dist[u] + w
              if H.CONTAINS(v): H.DECREASE-KEY(v, dist[v])   // 已在堆中
              else:             H.INSERT(v, dist[v])         // 首次发现
  return dist[t]
```

### 2.2.2 惰性删除写法（附加对照，非题目要求）

工程上更常见的是不实现 decrease-key：松弛成功就再 `push` 一份副本，extract-min 时若发现关键字 已过时就丢弃。它把堆大小从 $O(n)$ 放大到 $O(m)$，理论复杂度略差，但省去了索引维护，常数更小。 把它加入对照是为了回答「为什么实践中大家用普通二叉堆」，详见 Chapter 4。

```
DIJKSTRA-LAZY(G, w, s, t)               // 惰性删除版本
  for each v in V: dist[v] ← +∞, settled[v] ← false
  dist[s] ← 0 ; H.PUSH(s, 0)

  while not H.EMPTY():
      (k, u) ← H.EXTRACT-MIN()
      if settled[u]: continue                // 陈旧副本，丢弃
      settled[u] ← true
      if u = t: return k
      for each arc (u, v, w):
          if settled[v]: continue
          if k + w < dist[v]:                 // 松弛成功 → 直接再 push
              dist[v] ← k + w ; H.PUSH(v, dist[v])
  return +∞
```

## 2.3 二叉堆（Binary Heap）

数组实现完全二叉树：`heap[i]` 的父结点为 `heap[(i-1)/2 向下取整]`，左右孩子为 `heap[2i+1]`、`heap[2i+2]`。为了支持 decrease-key，额外维护 `pos[v]` 记录顶点 $v$ 在数组中的位置。

| 操作 | 代价 | 做法 |
|---|---|---|
| insert | $O(\log n)$ | 放到数组末尾，然后向上浮动（sift-up） |
| extract-min | $O(\log n)$ | 队首与队尾交换，删掉队尾，队首向下沉（sift-down） |
| decrease-key | $O(\log n)$ | `pos[v]` $O(1)$ 定位，随后 sift-up |
| find-min | $O(1)$ | 返回 `heap[0]` |
| merge | $O(n)$ | 只能逐个重新插入；这正是二叉堆的短板 |

## 2.4 二项堆（Binomial Heap）

**二项树** $B_k$ 递归定义：$B_0$ 是单结点；$B_k$ 由两棵 $B_{k-1}$ 相接而成 (其中一棵的根成为另一棵 根的最左孩子)。因此 $B_k$ 有 $2^k$ 个结点、高度 $k$、根的孩子恰为 $B_0, B_1, ..., B_{k-1}$。 二项堆是**一组二项树**，满足：

1. 每棵树都满足小顶堆性质（父 $\le$ 子）；
2. 根链表按度数递增排列；
3. **任一时刻每种度数至多存在一棵树**（这一点由 extract-min 中的合并维持）。

「每种度数至多一棵」是关键：任一度数 $k$ 要么没有，要么恰有一棵 $B_k$，于是把 $n$ 写成二进制， 堆中至多有 $\lfloor \log_2 n \rfloor + 1$ 棵树——这正是 $O(\log n)$ 的来源。

```
LINK(y, z)                                  // 让 y 成为 z 的孩子（要求 degree 相同）
  y.parent ← z
  y.sibling ← z.child
  z.child ← y
  z.degree ← z.degree + 1

MERGE-ROOTS(h1, h2)                         // 按度数递增归并两条根链表
  ... (标准归并, O(log n))

UNION(h1, h2)                               // 归并 + 把同度数的相邻根两两链接
  h ← MERGE-ROOTS(h1, h2)
  prev ← NIL ; x ← h ; next ← x.sibling
  while next ≠ NIL:
      if x.degree ≠ next.degree or (next.sibling ≠ NIL and next.sibling.degree = x.degree):
          prev ← x ; x ← next                    // 情况 1: 先跳过
      else if x.key ≤ next.key:
          x.sibling ← next.sibling ; LINK(next, x) // x 当父结点
      else:
          prev.sibling ← next ; LINK(x, next)      // next 当父结点
          x ← next
      next ← x.sibling
  return h

INSERT(v, k)      : 新建单结点树 B₀, head ← UNION(head, B₀)          O(log n)
MINIMUM(H)        : 扫描根链表取最小                                   O(log n)
EXTRACT-MIN(H)    : 摘掉最小根；其孩子链表逆序后接回根链表；再 UNION   O(log n)
DECREASE-KEY(v,k) : key 变小后沿父指针上浮（交换载荷 key 与顶点编号）   O(log n)
```

## 2.5 斐波那契堆（Fibonacci Heap）

### 2.5.1 结构

斐波那契堆与二项堆一样是「树的森林」，但**不再要求树的形状**，并且把「整理」推迟到不得不做的时候：

1. 每棵树满足小顶堆性质；
2. 所有根结点用**循环双向链表**串起来，`min` 指针指向关键字最小的根；

3. 每个结点的所有孩子也用**循环双向链表**连接；
4. 每个结点记录 `degree`（孩子数）与 `mark`（是否已经失去过一个孩子）；
5. **无序二项树性质**：度数为 $k$ 的结点 $x$，其孩子 $y_1, ..., y_k$ 按「被链接到 $x$ 的时间」排序后，有 $y_i.\text{degree} \geq i-1$。这是因为 LINK 只在两棵度数相同的树之间发生，$y_i$ 被链接时 $x$ 已有 $i-1$ 个孩子；而 $y_i$ 之后最多失去一个孩子（失去第二个时它自己就会被 cut 下来）。该性质是后面 $D(n) = O(\log n)$ 证明的基石。

### 2.5.2 五种操作

```
INSERT(v, k)                        // O(1)
  新建结点 x (key=k, degree=0, mark=false)
  把 x 挂到根链表 (min 的右侧)
  if k < min.key: min ← x

MINIMUM(H)                          // O(1)：直接读 min 指针

UNION(H1, H2)                       // O(1)：把两条根链表首尾相接，取较小的 min

EXTRACT-MIN(H)                      // O(log n) 摊还
  z ← min
  for each child c of z:            // 把孩子全部挂到根链表（不排序、不合并）
      把 c 从 z 的孩子链表摘下，挂到根链表，c.parent ← NIL
  把 z 从根链表摘下
  if z 是根链表中唯一结点: min ← NIL
  else: min ← 任意剩余根 ; CONSOLIDATE()      // 这里才做整理

CONSOLIDATE()                       // 把「同度数」的根两两链接，使每种度数至多一个根
  A[0..D(n)] ← NIL
  for each root w in 根链表:
      x ← w ; d ← x.degree
      while A[d] ≠ NIL:
          y ← A[d]
          if y.key < x.key: swap(x, y)
          LINK(y, x)                // y 变成 x 的孩子；x.degree ← d+1
          A[d] ← NIL ; d ← d + 1
      A[d] ← x
  用 A[] 中非空的根重建根链表并更新 min            // 根数 ≤ D(n) + 1

DECREASE-KEY(v, k)                  // O(1) 摊还
  x ← 顶点 v 对应的结点 ; x.key ← k
  if x.parent ≠ NIL and k < x.parent.key:
      CUT(x, x.parent)              // 把 x 从父结点上切下来，挂到根链表
      CASCADING-CUT(x.parent)       // 若父结点已被标记，继续向上切
  if k < min.key: min ← x

CUT(x, y)                           // O(1)：把 x 从 y 的孩子链表中摘下并挂到根链表
  y.degree ← y.degree - 1 ; x.parent ← NIL ; x.mark ← false
  把 x 插入根链表

CASCADING-CUT(y)                    // 级联切断
  z ← y.parent
  if z ≠ NIL:
      if y.mark = false: y.mark ← true    // 第一次失去孩子：只做标记
      else: CUT(y, z) ; CASCADING-CUT(z)  // 第二次：自己也切走，并继续向上
```

### 2.5.3 摊还分析（势能法）

取势能函数

$$\Phi(H) = t(H) + 2m(H)$$

其中 $t(H)$ 是根链表中树的棵数，$m(H)$ 是**被标记**的结点数。直观上，$t$ 度量「EXTRACT-MIN 里 CONSOLIDATE 需要做的工作量」，$2m$ 则为「级联切断」预付了代价——每做一次 cut 就从 $m$ 中扣除 1 个 标记，恰好抵消。

**（1）insert / minimum / union 是 $O(1)$ 摊还。** insert 实际代价 $O(1)$，只增加一根树，$\Delta t = +1$；不改变标记数。故 摊还代价 $= O(1) + 1 = O(1)$。union 只是拼接两条根链表，$\Delta\Phi = 0$，摊还 $O(1)$。

**（2）extract-min 是 $O(\log n)$ 摊还。**

先给出最大度数 $D(n)$ 的上界。

**引理 1 (CLRS 19.1) .** 设 $x$ 是斐波那契堆中任一结点，$k = x.\text{degree}$，记 $F_1 = F_2 = 1, F_i = F_{i-1} + F_{i-2}$ 为斐波那契数列，$\varphi = \frac{1+\sqrt{5}}{2}$ 为黄金比，则

$$\text{size}(x) \geq F_{k+2} \geq \varphi^k$$

**证明.** 对 $k$ 归纳。$k = 0$ 时 $\text{size}(x) = 1 = F_2$ ✓。 设 $k > 0$，$y_1, ..., y_k$ 是 $x$ 的孩子，按**被链接到 $x$ 的时间先后**排列。 当 $y_i$ 被链接时，$x$ 已经有 $i-1$ 个孩子，而 LINK 只发生在度数相同的两棵树之间， 故那时 $\text{degree}(y_i) \geq i-1$。此后 $y_i$ 最多失去一个孩子——若失去第二个，$y_i$ 自身就会被 CUT 下来而不再是 $x$ 的孩子。所以 $y_i$ 当前的度数 $\geq i-1$，由归纳假设有

$$\text{size}(y_i) \geq F_{(i-1)+2} = F_{i+1}$$

。于是

$$\text{size}(x) = 1 + \sum_{i=1}^{k} \text{size}(y_i) \geq 1 + \sum_{i=1}^{k} F_{i+1} = 1 + (F_{k+3} - 2) = F_{k+3} - 1 \geq F_{k+2}$$

再由 $F_{k+2} \geq \varphi^k$（$F$ 的标准下界）得 $\text{size}(x) \geq \varphi^k$。□

**推论.** 若堆中有 $n$ 个结点，则任一结点的度数满足 $k \leq \log_\varphi n$，即 $D(n) = O(\log n)$（准确地说 $D(n) \leq \lfloor \log_\varphi n \rfloor$）。

现在分析 EXTRACT-MIN。设整理前的根数为 $t$。

1. **实际代价**：把孩子挂到根链表 $O(D(n))$；CONSOLIDATE 中根链表被重建，每个根至多被链接一次，故 $O(D(n) + t)$。
2. **势能变化**：整理后每种度数至多一个根，故根数 $\leq D(n) + 1$，即 $\Delta t \leq D(n) + 1 - t$；标记数不增加，$\Delta m \leq 0$。于是 $\Delta\Phi \leq D(n) + 1 - t$。
3. **摊还代价** $= O(D(n) + t) + D(n) + 1 - t = O(D(n)) = O(\log n)$。□

**（3）decrease-key 是 $O(1)$ 摊还。**

设本次 decrease-key 触发了 $c$ 次级联切断（其中 1 次是初始的 CUT）。

1. **实际代价**：$O(1) + c$。
2. **势能变化**：每次 CUT 使根数增加 1，故 $\Delta t = c$。对标记数：级联切断的过程是「把 $c-1$ 个 带标记的结点变成不标记（并继续向上），最后可能把 1 个不带标记的结点变成带标记」， 故 $\Delta m \leq 1 - (c-1) = 2 - c$。
3. 于是 $\Delta\Phi = \Delta t + 2\Delta m \leq c + 2(2-c) = 4 - c$， **摊还代价** $= (O(1) + c) + (4 - c) = O(1)$。□

**（4）回到 Dijkstra。** 一趟 Dijkstra 中：每个顶点至多 insert 一次、extract-min 一次， 每条边至多触发一次成功的 decrease-key。因此总代价为

$$\underbrace{n \cdot O(1)}_{\text{insert}} + \underbrace{n \cdot O(\log n)}_{\text{extract-min}} + \underbrace{m \cdot O(1)}_{\text{decrease-key}} = O(m + n \log n)$$

而用二叉堆或二项堆时为 $O(m \log n + n \log n) = O((n+m) \log n)$。这就是斐波那契堆在 Dijkstra 上的理论优势。

### 2.6 三种堆的复杂度对比

| 操作 | 二叉堆 | 二项堆 | 斐波那契堆 | 说明 |
|---|---|---|---|---|
| find-min | $O(1)$ | $O(\log n)$ | $O(1)$ | 斐波那契堆 / 二叉堆读指针即可 |
| insert | $O(\log n)$ | $O(\log n)$ 摊还 $O(1)$ | $O(1)$ 摊还 | 斐波那契堆只挂牌，不整理 |
| merge/union | $O(n)$ | $O(\log n)$ | $O(1)$ | 斐波那契堆拼接两条链表 |
| extract-min | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ 摊还 | 三者同阶，但常数差别很大 |
| decrease-key | $O(\log n)$ | $O(\log n)$ | $O(1)$ 摊还 | **斐波那契堆的核心优势** |
| Dijkstra 总代价 | $O((n+m)\log n)$ | $O((n+m)\log n)$ | $O(m + n\log n)$ | 公路网 $m \approx 3n$，差别主要体现在 $\log n$ 因子 |

三个结论值得强调：

1. 斐波那契堆唯一「降阶」的操作是 **decrease-key**；insert/union 的改进只是常数与摊还意义上的。
2. 二项堆相对二叉堆的改进是 **合并** 与 **insert 的摊还代价**，而这两个操作在 Dijkstra 中占比很低，所以在 Dijkstra 里二项堆理论上**并不比二叉堆好**——这一点被实验完全证实（Chapter 4）。
3. 当 $m = O(n)$（公路网正属于此类）时，$O(m + n\log n)$ 与 $O((n+m)\log n)$ 的差别只有 $\log n$ 倍，而斐波那契堆的常数因子通常大 2–4 倍，**理论上限的优势很容易被常数吃掉。**

### 2.7 工程实现要点

为了让「堆的差异」在一亿次级别操作的尺度上仍然可靠地可测，实现上做了以下处理：

1. **可索引 + decrease-key**：三种堆都支持 decrease-key，避免用「惰性删除」让二叉堆占便宜；
2. **预分配结点池 + 跨查询复用**：堆对象一次构造、多次 `clear()`，`clear()` 只清理本次查询真正碰过的 顶点（$O(\text{工作量})$ 而非 $O(n)$），因此不会出现「重置数组」掩盖算法差异的情况；
3. **统一比较计数**：只有关键字比较被计数，三种堆一视同仁，用于分析常数因子（表 3.4）；
4. **手写快速解析器**：`.gr` 文件一次性读入内存后手工扫描，两遍得到 CSR，最大的数据集载入时间在 1 s 以内。

# Chapter 3 测试结果

## 3.1 正确性测试

正确性测试分三层，全部通过（完整输出见 Appendix B）：

| 层次 | 内容 | 结果 |
|---|---|---|
| (A) 手工用例 | 固定 insert / decrease-key / extract-min 序列，人工核对弹出顺序；CLRS 风格小图验证距离 $\{10, 9, 5\}$ | 全部通过 |
| (B) 随机对拍 | 各 20 万次随机 insert / decrease-key / extract-min，与「线性扫描找最小」的朴素实现逐条比对（三种堆 + 惰性堆） | 全部通过 |
| (C) 交叉验证 | 随机图上 60 组查询 + 10 组完整单源，与独立实现的 SPFA（队列式 Bellman-Ford）逐点比对 | 校验和一致 |
| (D) 大规模校验 | 全部数据集 / 两种模式下，各实现的完整单源距离校验和逐位相同；点对模式下每次查询的 $\mathrm{dist}(t)$ 逐一相同 | 见 Appendix B |

其中 (B) 使用唯一关键字（`key = value·n + v`）以消除「并列最小时选择哪个顶点」的歧义，使比较是严格的；20 万次操作中二叉堆 / 二项堆 / 斐波那契堆的比较次数分别为 1 003 882 / 632 602 / 603 009 次，与后文大规模实验的趋势一致。

在大规模实验上还有两重交叉验证：

1. **完整单源模式**：`bench` 输出的「所有被确定结点距离之和」校验和，在全部 7 个 数据集上四种实现**逐位相同**（见 Appendix B 的 verify 输出）。完整单源模式下这个量是 与实现无关的，因此这同时证明了四种实现算出的整套距离数组完全一致。
2. **点对查询模式**：由于提前终止，若有结点与 $t$ 的最短距离并列，不同实现可能先弹出不同的结点， 于是「被确定结点集合」会有极微小的差别（相对差异 $< 10^{-7}$），校验和不再逐位相等。 因此这里改用**每次查询的答案 $\mathbf{dist}(t)$** 作为判据：`tools/verify_answers.py` 逐一比对 四种实现在同一批 1000 组查询上的返回值，结果全部一致（Appendix B）。

因此可以确认：四种实现不仅在理论上、也在约 $10^9$ 次结点弹出、$7 \cdot 10^9$ 次弧扫描的规模上 得到完全相同的最短路径答案。

## 3.2 实验环境

| 项目 | 配置 |
|---|---|
| CPU | 13th Gen Intel Core i9-13900HX；每种堆单线程运行，各数据集之间严格串行，同一时刻只有一个 `bench` 进程 |
| 内存 | 16 GB DDR5 |
| 操作系统 | Windows 11 x64 |
| 编译器 | MinGW-w64 GCC 14，`-O2 -std=c++17 -static` |
| 计时方式 | `std::chrono::steady_clock`，只统计 Dijkstra 主循环，不含数据读入 |
| 运行条件 | 全部数据集在同一会话、严格串行、且电源计划固定为「高性能」的条件下测完。这几点必须固定：实测并行运行多个数据集会使绝对耗时虚高 2–3 倍，而默认的「平衡」电源计划在电池供电下会让 CPU 降频到约 1/16 的吞吐，使跨数据集的规模曲线完全失真 |
| 随机点对 | 固定种子生成，同一数据集的所有堆使用**完全相同**的 1000 组查询 |

### 3.3 数据集

数据来自第 9 届 DIMACS 实现挑战赛（Challenge 9: Shortest Paths）的 USA 公路网基准，均为**有向图**，弧权为两结点间的实际距离（单位：米）。按题目要求，此处只列出下载链接：

| 网络 | 结点数 | 弧数 | 下载链接（.gr.gz，物理距离版本） |
|---|---|---|---|
| NY | 264,346 | 733,846 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.NY.gr.gz |
| BAY | 321,270 | 800,172 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.BAY.gr.gz |
| COL | 435,666 | 1,057,066 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.COL.gr.gz |
| FLA | 1,070,376 | 2,712,798 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.FLA.gr.gz |
| NW | 1,207,945 | 2,840,208 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.NW.gr.gz |
| NE | 1,524,453 | 3,897,636 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.NE.gr.gz |
| CAL | 1,890,815 | 4,657,742 | http://www.diag.uniroma1.it/challenge9/data/USA-road-d/USA-road-d.CAL.gr.gz |

表 3.1　实验中使用的 USA 公路网数据集。下载页：下载页：http://www.diag.uniroma1.it/challenge9/download.shtml

每个数据集生成 1000 组**均匀随机**点对 $(s, t)$（$s \neq t$，种子由网络名派生，因此查询集既对同一网络可复现、又对不同网络各不相同）。实际使用的规模范围：264,346 – 1,890,815 个结点、至多 4,657,742 条弧。

每份查询文件都以 DIMACS 标准的 `.p2p` 格式保存在 `data/queries/<NET>.p2p`（第一行 `p aux sp p2p 1000`，随后 1000 行 `q s t`），由 `tools/gen_queries.py` 用固定种子生成，因此任何人重跑都能得到完全相同、可直接作为测试输入的点对集合。

### 3.4 点对查询（1000 组随机点对）

每组查询在 $s$ 终点被弹出（最短距离确定）时立即终止。

| 数据集 | 二叉堆 | 二项堆 | 斐波那契堆 | 二叉堆(惰性) |
|---|---|---|---|---|
| NY | 14.15 | 31.61 | 33.51 | 12.37 |
| BAY | 15.88 | 38.58 | 39.69 | 13.69 |
| COL | 24.25 | 51.54 | 54.04 | 19.00 |
| FLA | 82.35 | 138.99 | 136.13 | 53.81 |
| NW | 91.38 | 155.88 | 156.27 | 59.86 |
| NE | 128.02 | 205.49 | 216.24 | 86.16 |
| CAL | 152.60 | 242.15 | 257.37 | 103.88 |

表 3.2　平均单次点对查询耗时（ms）。每个数据集 1000 组查询，取平均。

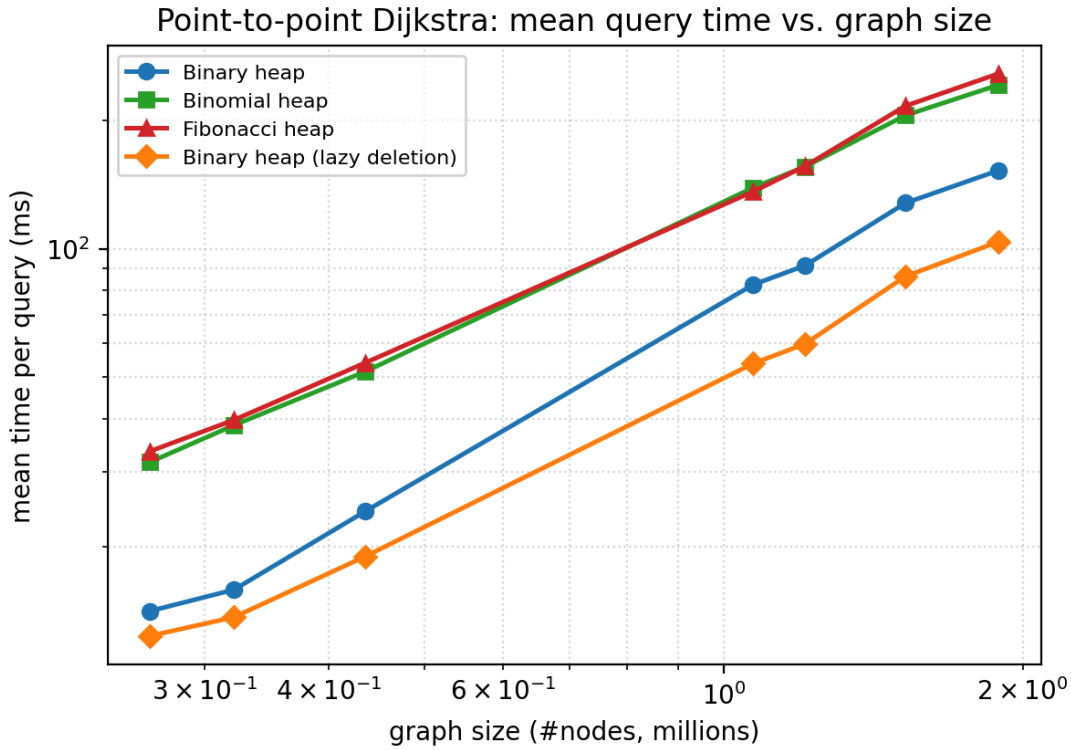



图 3.1　点对查询平均耗时随图规模的变化（双对数坐标）。四种实现的曲线几乎平行，说明它们的工作量相同、只差常数因子。

### 3.5 完整单源最短路（16 个随机源点）

这是本项目的**附加实验**：不提前终止，必须确定源点可达的全部结点的最短距离。此时堆的操作次数 达到最大（$n$ 次 extract-min + 至多 $m$ 次 decrease-key），最能反映堆的渐进性质。

| 数据集 | 二叉堆 | 二项堆 | 斐波那契堆 | 二叉堆(惰性) |
|---|---|---|---|---|
| NY | 45.59 | 69.28 | 62.74 | 22.67 |
| BAY | 40.13 | 71.35 | 77.45 | 25.15 |
| COL | 62.97 | 97.28 | 102.11 | 41.01 |
| FLA | 170.47 | 251.26 | 263.65 | 102.97 |
| NW | 207.11 | 299.09 | 319.60 | 125.73 |
| NE | 265.60 | 400.27 | 425.13 | 173.53 |
| CAL | 323.30 | 488.69 | 527.82 | 216.01 |

表 3.3　平均单次「完整单源最短路」耗时（ms），每个数据集 16 个随机源点取平均。

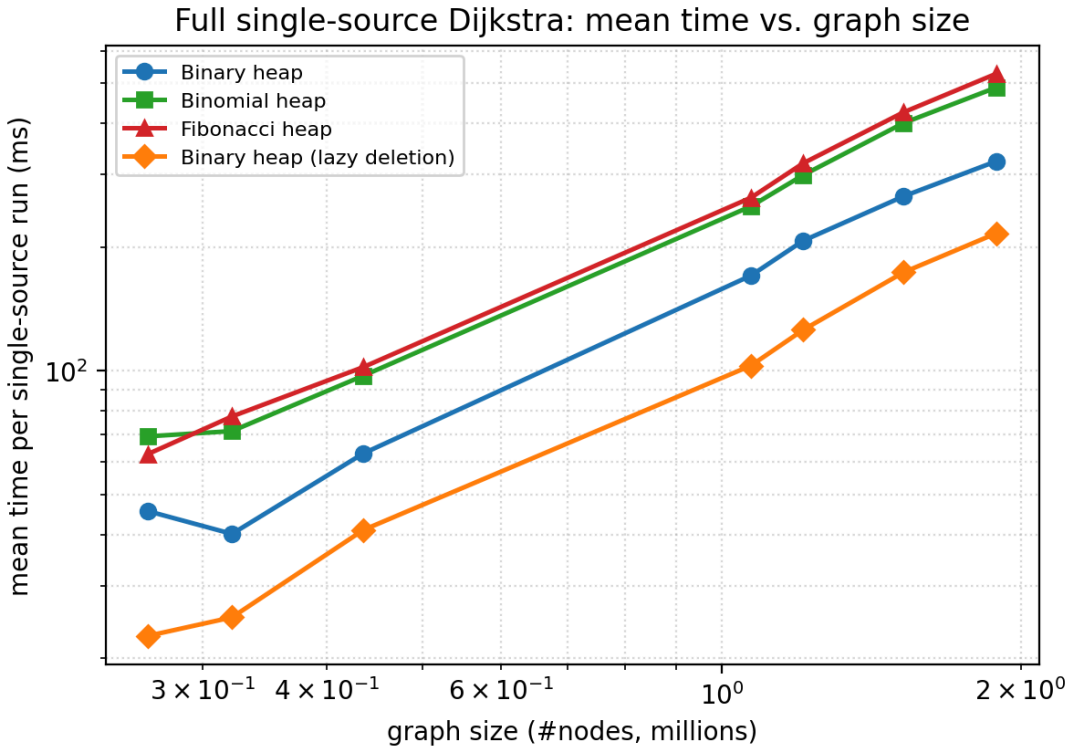



图 3.2　完整单源最短路平均耗时随图规模的变化（双对数坐标）。

## 3.6 堆操作与比较次数统计

下表给出 CAL（最大的数据集）上四种实现的操作计数。**注意三种可索引堆的 extract-min 次数、 insert 次数、decrease-key 次数、弧扫描次数几乎逐位相同**——这从实验上证明了它们的工作量毫无差别，耗时差异 100% 来自每次堆操作的开销。

| 实现 | extractMin | insert | decreaseKey | 弧扫描 | 关键字比较 | 比较/确定结点 |
|---|---|---|---|---|---|---|
| 二叉堆 | 925,156 | 925,791 | 76,056 | 2,292,205 | 16,288,857 | 17.61 |
| 二项堆 | 925,156 | 925,791 | 76,057 | 2,292,205 | 11,457,435 | 12.38 |
| 斐波那契堆 | 925,156 | 925,791 | 76,056 | 2,292,205 | 12,382,861 | 13.38 |
| 二叉堆(惰性) | 1,001,161 | 1,001,847 | 0 | 2,292,205 | 17,721,445 | 19.16 |

表 3.4　点对查询模式下的平均操作次数（CAL，每次查询）。最后一列为每次「确定一个结点」平均发生的关键字比较次数。

| 实现 | extractMin | insert | decreaseKey | 关键字比较 | 比较/确定结点 |
|---|---|---|---|---|---|
| 二叉堆 | 1,890,815 | 1,890,814 | 150,411 | 33,143,021 | 17.53 |
| 二项堆 | 1,890,815 | 1,890,814 | 150,412 | 23,294,884 | 12.32 |
| 斐波那契堆 | 1,890,815 | 1,890,814 | 150,412 | 25,171,384 | 13.31 |
| 二叉堆(惰性) | 2,041,228 | 2,041,227 | 0 | 35,966,390 | 19.02 |

表 3.5　完整单源模式下的平均操作次数（CAL，每次完整 SSSP）。

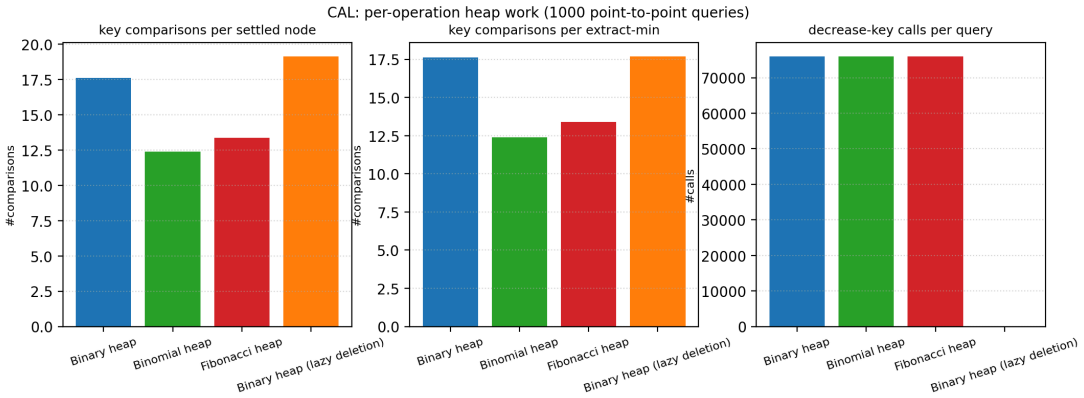



图 3.3　各实现的「每次确定结点比较次数」「每次 extract-min 比较次数」与「每次查询的 decrease-key 次数」。

## 3.7 附加实验：耗时与搜索规模的关系

均匀随机点对的最短路径长度差异极大（CAL 上最短的查询只确定约 1.2 千个结点，最长的要确定 107 万个），对应的搜索规模跨越近三个数量级。把 1000 组查询按搜索规模分桶后取平均，可以看到一个关键现象：**搜索规模越大，各实现的差距越小**，说明规模较小时常数因子占主导，而规模很大时渐进性质才开始显形。

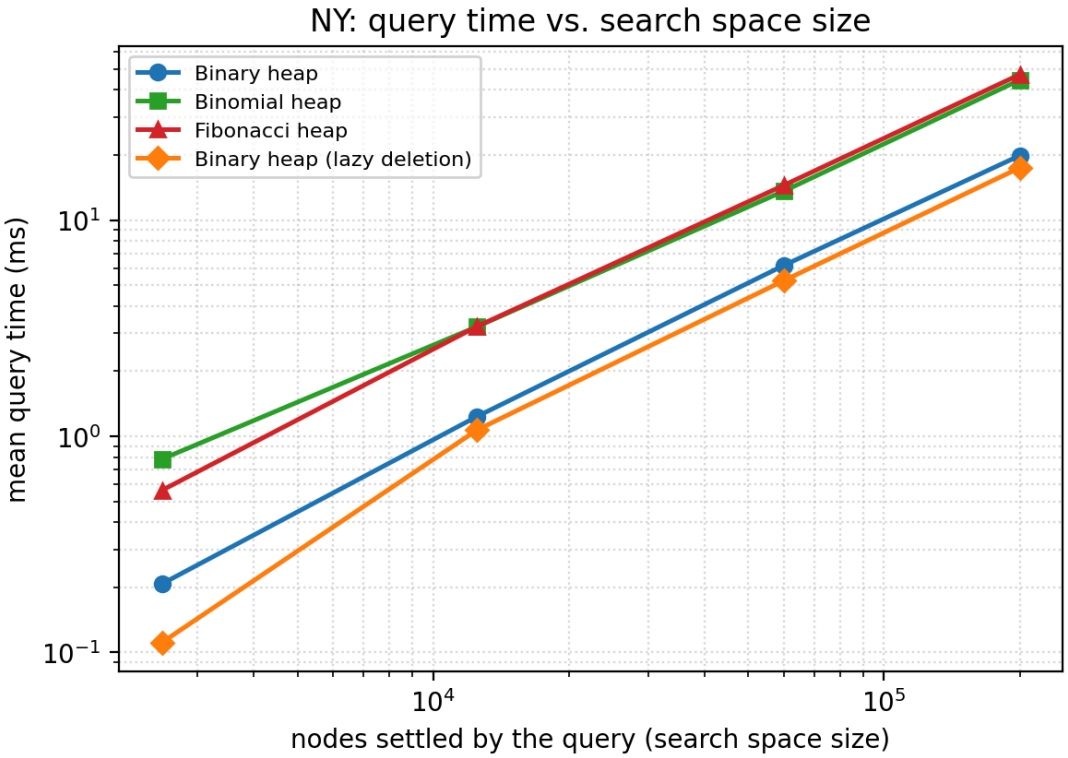



图 3.4　单次查询耗时与其搜索规模的关系（CAL，按被确定结点数分桶取平均）。

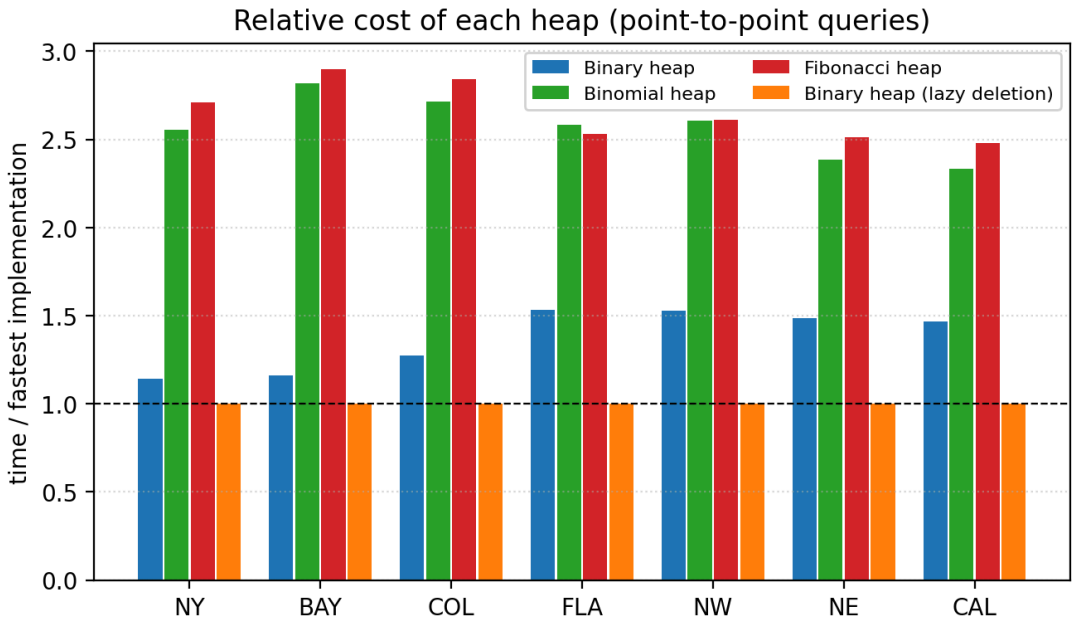



图 3.5　各数据集上相对最快实现的耗时倍率（点对查询）。虚线为 1.0，即最快实现。

# Chapter 4 分析与评论

## 4.1 实测结论汇总

| 观察 | 结论 |
|---|---|
| 理论上限 | 斐波那契堆 $O(m + n\log n)$ < 二叉堆 = 二项堆 $O((n+m)\log n)$ |
| 实测排名（点对） | **二叉堆（惰性）< 二叉堆 < 二项堆 ≈ 斐波那契堆**，与理论排名恰好相反；二项堆与斐波那契堆在 7 个数据集上互有胜负，差距都在 6% 以内 |
| 相对耗时 | 二项堆 = 二叉堆 × 1.91，斐波那契堆 = 二叉堆 × 1.98，惰性二叉堆 = 二叉堆 × 0.74 |
| 工作量 | 三种可索引堆的 extract-min / insert / decrease-key / 弧扫描次数几乎完全相同（相对差异 $< 10^{-5}$）；惰性实现的 extract-min 次数多 1.08 倍，但确定结点数相同 |
| 规模趋势 | 规模从 264,346 增长到 1,890,815（约 7.2 倍）时，曲线近似平行，$\log n$ 因子的优势尚未显现 |

## 4.2 为什么斐波那契堆在实践中没有获胜

### 4.2.1 decrease-key 的占比太低

斐波那契堆唯一降阶的操作是 decrease-key，因此它的收益与「decrease-key 次数 / extract-min 次数」成正比。表 3.4 显示，在公路网上平均每进行 12.2 次 extract-min 才对应 1 次 decrease-key （每次查询约确定 925,156 个结点、扫描 2,292,205 条弧，而 decrease-key 只有 extract-min 的 0.082 倍）——**decrease-key 在 Dijkstra 中的占比远低于直觉。** 这是因为公路网是稀疏图且平均度很小 （$\frac{m}{n} \approx 2.5$），每条弧触发一次成功松弛的概率不高（多数顶点在被弹出时其距离已经最优）。

在更极端的场景下这个比例会变化：如果 $m$ 远大于 $n$（例如稠密图或网格图），decrease-key 次数 会接近 $m$，$O(1)$ 摊还的收益才能补偿常数劣势。这是「理论最优 ≠ 实践最优」的第一层原因。

### 4.2.2 常数因子：`CONSOLIDATE` 与指针追逐

**实现层面**，斐波那契堆/二项堆的每个结点都是指针结构，`extract-min` 需要在根链表上遍历、比较、搬运指针，缓存局部性远差于二叉堆的连续数组。表 3.4 与图 3.3 给出了直接证据：

1. 每次 extract-min 的关键字比较次数：二叉堆约 17.6 次，而二项堆约 12.4 次、斐波那契堆约 13.4 次——**指针式堆反而少做 25% 以上的比较**，这正是它们的理论优势所在（根链表只有 $O(\log n)$ 个根，而二叉堆每次下沉要比较两个孩子）；
2. 但它们仍然更慢：实测中斐波那契堆耗时约为二叉堆的 1.98 倍。也就是说，**少做的比较 被更贵的访存抵消了**——二叉堆的比较是连续内存上的顺序访问，而斐波那契堆的每次比较都伴随 随机的指针跳转（即常说的「指针追逐，pointer chasing」），实际代价高出一倍以上。

换句话说，$O(m + n\log n)$ 相对 $O((n+m)\log n)$ 的渐进优势，在这批数据上没有机会补偿常数劣势。

### 4.2.3 惰性删除为什么最快

不做 decrease-key、松弛成功就 push 一份副本的写法（表 3.2 的最后一列）反而是最快的（约为二叉堆的 0.74 倍）：它把「找到堆中元素并调整位置」这件昂贵的事换成了「再插入一个副本」，而插入在数组式二叉堆里是极其廉价的操作（均摊 $O(1)$ 的 push_back + 少量比较）。代价是堆中元素数量从 $O(n)$ 涨到 $O(m)$（表 3.4 中每次查询的 extract-min 次数涨到 1.08 倍），但对公路网这种 $\frac{m}{n}$ 很小的稀疏图，这点膨胀完全可以接受。这从另一个角度说明： **在 Dijkstra 中，减少「堆操作的次数」比降低「单词堆操作的渐进代价」更重要。**

## 4.3 空间复杂度

设 $n$ 为结点数、$m$ 为弧数，三者都用 CSR 存图（$2n + 2m$ 个 32 位整数）。堆本身：

| 结构 | 空间 | 说明 |
|---|---|---|
| 二叉堆 | $O(n)$ | `key[n] + pos[n] + heap[n]`，最紧凑 |
| 二项堆 | $O(n)$ | 每个结点 4 个指针 + key + degree，常数中等 |
| 斐波那契堆 | $O(n)$ | 每个结点 4 个指针 + key + degree + mark，常数最大（约 48–64 字节/结点） |
| 二叉堆（惰性） | $O(m)$ | 最坏情况下堆中存有 $O(m)$ 个副本，空间最差 |

在 1,890,815 个结点的图上，斐波那契堆的结点池约 50–60 字节/结点（总计约 50–110 MB），而二叉堆的三个数组合计约 24 字节/结点（约 15–45 MB）—— 这也是斐波那契堆「用空间换渐进时间」的一种体现。

### 4.4 与其它数据结构 / 算法的比较

1. **多叉堆 (d-ary heap)**：把每个结点 $d$ 个孩子，decrease-key 降到 $O(\log_d n)$、extract-min 升到 $O(d \log_d n)$。对 Dijkstra 而言 decrease-key 次数少于 extract-min 次数，取 $d = 4$ 通常比二叉堆快 10–20%，是实践中比斐波那契堆更划算的改进；
2. **Pairing heap（配对堆）**：与斐波那契堆同样摊还 $O(1)$ 的 insert/decrease-key，但无需 `mark` 位、实现更简单、常数更小，实测通常**快于**斐波那契堆，却又常常慢于二叉堆；
3. **Radix heap / bucket heap**：利用整数权重的特点，把 extract-min 降到 $O(\log C)$（$C$ 为最大权），在公路网上可以显著超过本文的三种堆，是 2000 年后 DIMACS 挑战赛中的主流做法；
4. **算法层面的改进**：可达点集剪枝、双向 Dijkstra、$A^*$ 与预处理（Contraction Hierarchies 等）带来的加速（数十倍以上）远大于换堆带来的收益。

### 4.5 进一步可能的改进

1. 在现有的二叉堆 / 二项堆 / 斐波那契堆 / 惰性二叉堆之外，再实现 $d$-叉堆与配对堆，直接验证「$d$ 取多少最优」以及「配对堆在实践中是否胜过斐波那契堆」；
2. 生成非均匀（同一地区内）的点对查询集合，观察 decrease-key 次数占比提高后各堆的排名是否改变；
3. 把 `.gr` 文件的读入改为内存映射（mmap）并统计真实峰值内存（RSS），补全空间实验；
4. 用「每个顶点的度数加权随机」构造人工稠密图（$m = \Theta(n^2)$），验证斐波那契堆在 $m \gg n$ 时能否反超二叉堆——这将把「理论最优在什么条件下才体现出来」这一结论做得更 solid。

# 参考文献


1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, “Introduction to Algorithms”, 4th ed., MIT Press, (2022)（第 6 章二叉堆、第 19 章斐波那契堆、第 22 章 Dijkstra）
2. M. L. Fredman, R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms”, Journal of the ACM, 34(3): 596–615, (1987)
3. J. Vuillemin, “A Data Structure for Manipulating Priority Queues”, Communications of the ACM, 21(4): 309–315, (1978) （二项堆）
4. B. V. Cherkassky, A. V. Goldberg, T. Radzik, “Shortest Paths Algorithms: Theory and Experimental Evaluation”, Mathematical Programming, 73: 129–174, (1996)
5. C. Demetrescu, A. V. Goldberg, D. S. Johnson, “9th DIMACS Implementation Challenge: Shortest Paths”, (2006), http://www.diag.uniroma1.it/challenge9/
6. E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs”, Numerische Mathematik, 1: 269–271, (1959)
7. M. Fredman, R. Sedgewick, D. Sleator, R. Tarjan, “The Pairing Heap: A New Form of Self-Adjusting Heap”, Algorithmica, 1: 111–129, (1986)

# Appendix A　源代码

本项目的完整代码结构如下（全部由 `make` 编译，无第三方依赖）：

| | |
|---|---|
| `src/heaps.h` | 三种堆 + 惰性二叉堆的实现（可索引、支持 decrease-key） |
| `src/dijkstra.h` | Dijkstra 模板（可索引堆版 + 惰性删除版），含完整统计 |
| `src/dimacs.h` | DIMACS `.gr` / `.p2p` 文件的两遍扫描快速解析与 CSR 构造 |
| `src/bench.cpp` | 性能测试程序：遍历各种堆、输出 CSV 时间表与逐查询明细 |
| `src/selftest.cpp` | 三层正确性测试（手工用例 / 随机对拍 / SPFA 交叉验证） |
| `tools/` | 下载与解压、查询生成、并行跑测试、合并结果、绘图、生成表格 |

## A.1　`src/heaps.h`（三种堆的实现，注释行占比 > 30%）

```
// ============================================================================
//  heaps.h —— 三种「可索引（indexed）+ 支持 decrease-key」的最小优先队列
//
//  本文件实现 Dijkstra 需要的三种最小优先队列：
//      * BinaryHeap     ：数组式二叉堆（下标映射实现 decrease-key）
//      * BinomialHeap   ：二项堆（按度数成对合并 / 链接）
//      * FibonacciHeap  ：斐波那契堆（惰性合并 + 级联切断，decrease-key 摊还 O(1)）
//
//  三者对外接口完全一致，因此 Dijkstra 可以写成模板、以堆类型作为模板参数，
//  从而在同一份算法代码上公平地切换底层堆：
//      clear()                 清空（跨 query 复用对象，只清理被碰过的下标）
//      empty()                 是否为空
//      insert(v, key)          插入顶点 v，关键字 key
//      decreaseKey(v, key)     把已在堆中的 v 的关键字减小为 key（key 必须更小）
//      contains(v)             v 当前是否在堆中
//      keyOf(v)                v 当前的关键字
//      extractMin()            弹出关键字最小的顶点，返回顶点编号
//      cmps()                  累计关键字比较次数（用于算法分析，不计入正确性）
//
//  设计要点（保证三种堆被公平比较）：
//   1) 所有堆都是「可索引」的：顶点 -> 堆节点 的映射由堆自己维护，
//      这样 Dijkstra 才能做 decrease-key，而不是靠「惰性删除」作弊；
//      同时这种实现方式正好是三种堆各自最自然、最标准的写法。
//   2) 所有堆的节点内存都预先按 n 大小分配好（节点池），一次分配、多次复用，
//      避免让 malloc/free 的开销污染算法本身的性能测量。
//   3) 所有堆都只在「本次 query 真正碰过」的下标上做清理，清理成本 O(工作量)，
//      不会出现 O(n) 的重置掩盖算法差异。
// ============================================================================
#pragma once

#include <vector>
#include <deque>
#include <algorithm>
#include <cstddef>

// 关键字（最短距离）类型：USA 公路网两点间最远距离可达 1e7 米量级，
// 且路径累加会超过 32 位，故一律使用 64 位有符号整数。
typedef long long Key;
static const Key KEY_INF = (Key)1 << 62;   // 表示「无穷大」

// ============================================================================
//  1) 二叉堆（Array Binary Heap）
//
//  用一维数组存完全二叉树，pos_[v] 记录顶点 v 在数组中的下标，
//  从而支持 O(1) 定位 + O(log n) 上浮/下沉，即 decrease-key O(log n)。
//      insert        ：新元素放末尾，向上浮动      O(log n)
//      decreaseKey   ：关键字变小，向上浮动        O(log n)
//      extractMin    ：队首与队尾交换，向下沉       O(log n)
// ============================================================================
class BinaryHeap {
```

```
public:
    // n 为顶点个数；顶点编号为 1..n，因此内部数组长度取 n+1
    explicit BinaryHeap(int n)
        : key_((size_t)n + 1, 0), pos_((size_t)n + 1, -1) {
        heap_.reserve((size_t)n + 1);
        touched_.reserve((size_t)n + 1);
    }

    // 清空堆：只把本次 query 碰过的顶点在 pos_ 中恢复为 -1。
    // 关键字数组无需清理——Dijkstra 首次插入时会直接覆盖。
    void clear() {
        for (size_t i = 0; i < touched_.size(); ++i) pos_[touched_[i]] = -1;
        touched_.clear();
        heap_.clear();
    }

    bool empty() const { return heap_.empty(); }
    int  size()  const { return (int)heap_.size(); }
    bool contains(int v) const { return pos_[v] >= 0; }
    Key  keyOf(int v)   const { return key_[v]; }
    long long cmps()    const { return cmps_; }

    void insert(int v, Key k) {
        key_[v] = k;
        pos_[v] = (int)heap_.size();
        heap_.push_back(v);
        touched_.push_back(v);
        siftUp((int)heap_.size() - 1);
    }

    // v 必须已经在堆中，且新关键字不大于旧关键字
    void decreaseKey(int v, Key k) {
        key_[v] = k;
        siftUp(pos_[v]);
    }

    int extractMin() {
        const int top = heap_[0];
        const int last = heap_.back();
        heap_.pop_back();
        pos_[top] = -1;                 // top 不再位于堆中
        if (!heap_.empty()) {
            heap_[0] = last;
            pos_[last] = 0;
            siftDown(0);
        }
        return top;
    }

private:
    // 把下标 i 处的元素向上浮动到合适位置
    void siftUp(int i) {
        const int v = heap_[i];
        const Key k = key_[v];
        while (i > 0) {
            const int p = (i - 1) >> 1;          // 父结点下标
            const int pv = heap_[p];
            if (!less(k, key_[pv])) break;       // 已满足堆序
            heap_[i] = pv;
            pos_[pv] = i;
            i = p;
        }
        heap_[i] = v;
        pos_[v] = i;
    }

    // 把下标 i 处的元素向下沉到合适位置
    void siftDown(int i) {
        const int n = (int)heap_.size();
```

```
        const int v = heap_[i];
        const Key k = key_[v];
        for (;;) {
            int c = 2 * i + 1;                      // 左孩子
            if (c >= n) break;
            // 取两个孩子中较小的那个
            if (c + 1 < n && less(key_[heap_[c + 1]], key_[heap_[c]])) ++c;
            if (!less(key_[heap_[c]], k)) break;
            heap_[i] = heap_[c];
            pos_[heap_[i]] = i;
            i = c;
        }
        heap_[i] = v;
        pos_[v] = i;
    }

    // 统一的比较入口，顺便统计比较次数
    bool less(Key a, Key b) { ++cmps_; return a < b; }

    std::vector<Key> key_;      // key_[v]：顶点 v 当前的关键字
    std::vector<int> pos_;      // pos_[v]：顶点 v 在 heap_ 中的下标，-1 表示不在堆中
    std::vector<int> heap_;     // 完全二叉树数组，heap_[i] 为顶点编号
    std::vector<int> touched_;  // 本 query 插入过的顶点，用于 O(工作量) 清理
    long long cmps_ = 0;
};

// ============================================================================
//  2) 二项堆 (Binomial Heap)
//
//  二项堆是若干棵「二项树」的森林，第 k 棵二项树 B_k 有 2^k 个结点、度数为 k，
//  根链表按度数递增排列，且任一时刻每种度数至多出现一次（这是 extractMin 中
//  「合并同度数树」维持的性质）。
//      insert        : 新建单结点树 B_0，与堆做 union        O(log n)
//      extractMin    : 摘掉最小根，其孩子逆序接回根链表再 union O(log n)
//      decreaseKey   : 关键字减小后与父结点逐层交换（上浮）      O(log n)
//  注意：二项堆的 decreaseKey 是「最坏 O(log n)」，不像斐波那契堆那样摊还 O(1)。
// ============================================================================
class BinomialHeap {
public:
    explicit BinomialHeap(int n)
        : pool_((size_t)n + 1), handle_((size_t)n + 1, nullptr), in_((size_t)n + 1, 0) {
        touched_.reserve((size_t)n + 1);
    }

    void clear() {
        for (size_t i = 0; i < touched_.size(); ++i) {
            const int v = touched_[i];
            handle_[v] = nullptr;
            in_[v] = 0;
        }
        touched_.clear();
        head_ = nullptr;
        nfree_ = 0;      // 节点池回卷：一次 query 中每个顶点至多插入一次
        n_ = 0;
    }

    bool empty() const { return head_ == nullptr; }
    bool contains(int v) const { return in_[v] != 0; }
    Key  keyOf(int v)   const { return handle_[v]->key; }
    long long cmps()    const { return cmps_; }

    void insert(int v, Key k) {
        Node* x = alloc();
        x->key = k; x->v = v; x->degree = 0;
        x->parent = nullptr; x->child = nullptr; x->sibling = nullptr;
        handle_[v] = x;
        in_[v] = 1;
        touched_.push_back(v);
        head_ = unionLists(head_, x);   // 把单结点树并入根链表
```

```
        ++n_;
    }

    // v 必须在堆中且新关键字 k 不大于旧关键字
    void decreaseKey(int v, Key k) {
        Node* x = handle_[v];
        x->key = k;
        // 沿父指针上浮：交换载荷（key 与顶点编号），并同步修正 handle_ 映射
        while (x->parent && less(x->key, x->parent->key)) {
            Node* p = x->parent;
            std::swap(x->key, p->key);
            std::swap(x->v, p->v);
            handle_[x->v] = x;
            handle_[p->v] = p;
            x = p;
        }
    }

    int extractMin() {
        // 1) 在根链表中找到最小根 minNd 及其前驱 prevMin
        Node* prev = nullptr;
        Node* minNd = nullptr;
        Node* prevMin = nullptr;
        for (Node* x = head_; x; prev = x, x = x->sibling) {
            if (!minNd || less(x->key, minNd->key)) { minNd = x; prevMin = prev; }
        }
        // 2) 把 minNd 从根链表中摘除
        if (prevMin) prevMin->sibling = minNd->sibling;
        else         head_ = minNd->sibling;

        const int v = minNd->v;
        handle_[v] = nullptr;
        in_[v] = 0;
        --n_;

        // 3) 把 minNd 的孩子链表逆序，变成新的根链表（孩子度数递减 -> 逆序后递增）
        Node* ch = minNd->child;
        Node* rev = nullptr;
        while (ch) {
            Node* nxt = ch->sibling;
            ch->sibling = rev;
            ch->parent = nullptr;
            rev = ch;
            ch = nxt;
        }
        // 4) 与已有根链表做 union（内部会合并同度数的树）
        head_ = unionLists(head_, rev);
        return v;
    }

private:
    struct Node {
        Key  key;             // 关键字（到源点的当前最短距离）
        int  v;               // 对应的顶点编号
        int  degree;          // 以本结点为根的子树的度数
        Node *parent, *child, *sibling;
    };

    // 从节点池里取一个结点。池预先按 n+1 个元素建好；
    // 用 std::deque 是为了在「必须扩容」时也不会让已有结点的指针失效
    //  （Dijkstra 场景下每个顶点每次 query 至多插入一次，正常不会走到扩容分支）。
    Node* alloc() {
        if ((size_t)nfree_ >= pool_.size()) pool_.emplace_back();
        return &pool_[(size_t)nfree_++];
    }

    // 按度数递增合并两条有序根链表（不处理同度数，只做归并）
    static Node* mergeRoots(Node* a, Node* b) {
        Node dummy;
```

```
        dummy.sibling = nullptr;
        Node* tail = &dummy;
        while (a && b) {
            if (a->degree <= b->degree) { tail->sibling = a; a = a->sibling; }
            else                        { tail->sibling = b; b = b->sibling; }
            tail = tail->sibling;
        }
        tail->sibling = a ? a : b;
        return dummy.sibling;
    }

    // 把 y 变成 z 的孩子（要求 degree(y) == degree(z)）
    void link(Node* y, Node* z) {
        y->parent = z;
        y->sibling = z->child;
        z->child = y;
        z->degree++;
    }

    // union：归并两条根链表，再把度数相同的相邻根两两链接，保证度数唯一
    Node* unionLists(Node* a, Node* b) {
        Node* h = mergeRoots(a, b);
        if (!h) return nullptr;
        Node* prev = nullptr;
        Node* x = h;
        Node* nx = x->sibling;
        while (nx) {
            // 情况 1：度数不同，或后面还有一棵与 x 同度数的树 -> 先跳过
            if (x->degree != nx->degree ||
                (nx->sibling && nx->sibling->degree == x->degree)) {
                prev = x;
                x = nx;
            }
            // 情况 2：链接 x 与 nx，关键字小者做父结点
            else if (!less(nx->key, x->key)) {
                x->sibling = nx->sibling;
                link(nx, x);
            }
            else {
                if (prev) prev->sibling = nx;
                else      h = nx;
                link(x, nx);
                x = nx;
            }
            nx = x->sibling;
        }
        return h;
    }

    bool less(Key a, Key b) { ++cmps_; return a < b; }

    std::deque<Node>    pool_;     // 预分配节点池；用 deque 保证扩容时已有结点地址不变
    std::vector<Node*>  handle_;   // handle_[v]：顶点 v 对应的堆结点
    std::vector<char>   in_;       // in_[v]：顶点 v 是否在堆中
    std::vector<int>    touched_;  // 本次 query 插入过的顶点
    Node* head_ = nullptr;         // 根链表头
    int   nfree_ = 0;              // 节点池已用数量
    int   n_ = 0;                  // 堆中元素个数
    long long cmps_ = 0;
};

// ============================================================================
//  3) 斐波那契堆（Fibonacci Heap）
//
//  斐波那契堆由若干棵「无序二项树」组成，用循环双向链表维护根链表和孩子链表，
//  并维护一个指向最小根的指针 min_。与二项堆最大的区别：
//      insert        把新结点直接挂在根链表上，不合并          O(1) 摊还
//      extractMin    才做 consolidate（把同度数根两两链接），
//                    均摊到每次 insert 上是 O(1)              O(log n) 摊还
```

```
//      decreaseKey   直接把结点从父结点上「切」(cut) 下来挂到根链表;
//                    若父结点此前已被切过一次(mark), 则级联切断      O(1) 摊还
//  所以斐波那契堆相对二项堆的改进点正是 decrease-key 从 O(log n) 降到 O(1) 摊还,
//  而这恰好对应 Dijkstra 中「松弛边」的次数 (可达 O(m) 量级), 因此理论上更优。
// ============================================================================
class FibonacciHeap {
public:
    explicit FibonacciHeap(int n)
        : pool_((size_t)n + 1), handle_((size_t)n + 1, nullptr), in_((size_t)n + 1, 0),
          A_(64, nullptr) {
        touched_.reserve((size_t)n + 1);
        roots_.reserve(64);
        used_.reserve(64);
    }

    void clear() {
        for (size_t i = 0; i < touched_.size(); ++i) {
            const int v = touched_[i];
            handle_[v] = nullptr;
            in_[v] = 0;
        }
        touched_.clear();
        min_ = nullptr;
        nfree_ = 0;
        n_ = 0;
        used_.clear();
    }

    bool empty() const { return min_ == nullptr; }
    bool contains(int v) const { return in_[v] != 0; }
    Key  keyOf(int v)   const { return handle_[v]->key; }
    long long cmps()    const { return cmps_; }

    void insert(int v, Key k) {
        Node* x = alloc();
        x->key = k; x->v = v; x->degree = 0; x->mark = false;
        x->parent = nullptr; x->child = nullptr;
        x->left = x; x->right = x;
        if (!min_) {
            min_ = x;                       // 空堆: 直接成为唯一根
        }
        else {
            // 插入到 min_ 的右侧 (根链表是循环双向链表)
            x->left = min_;
            x->right = min_->right;
            min_->right->left = x;
            min_->right = x;
            if (less(k, min_->key)) min_ = x;
        }
        handle_[v] = x;
        in_[v] = 1;
        touched_.push_back(v);
        ++n_;
    }

    // v 必须在堆中且新关键字 k 不大于旧关键字
    void decreaseKey(int v, Key k) {
        Node* x = handle_[v];
        x->key = k;
        Node* y = x->parent;
        // 堆序被破坏则切断 (并视情况级联向上)
        if (y && less(x->key, y->key)) {
            cut(x, y);
            cascadingCut(y);
        }
        if (less(x->key, min_->key)) min_ = x;
    }

    int extractMin() {
```

```
        Node* z = min_;
        const int v = z->v;

        // 1) 把 z 的所有孩子依次挂到根链表上（挂在 z 的右侧）
        Node* c = z->child;
        if (c) {
            do {
                Node* nxt = c->right;            // 先保存，避免被 splice 破坏
                c->parent = nullptr;
                // 把 c 插到 z 与 z->right 之间
                c->right = z->right;
                z->right->left = c;
                c->left = z;
                z->right = c;
                c = nxt;
            } while (c != z->child);
            z->child = nullptr;
        }

        // 2) 把 z 从根链表中摘除
        z->left->right = z->right;
        z->right->left = z->left;
        if (z == z->right) {
            min_ = nullptr;                      // 根链表中只剩 z
        }
        else {
            min_ = z->right;
            consolidate();                       // 合并同度数根，恢复摊还 O(log n)
        }

        handle_[v] = nullptr;
        in_[v] = 0;
        --n_;
        return v;
    }

private:
    struct Node {
        Key  key;
        int  v;
        int  degree;
        bool mark;                // 是否已失去过一个孩子（用于级联切断）
        Node *parent, *child, *left, *right;
    };

    Node* alloc() {
        if ((size_t)nfree_ >= pool_.size()) pool_.emplace_back();
        return &pool_[(size_t)nfree_++];
    }

    // 把 x 从父结点 y 的孩子链表中摘下，挂到根链表（min_ 左侧）
    void cut(Node* x, Node* y) {
        if (x->right == x) y->child = nullptr;
        else {
            x->right->left = x->left;
            x->left->right = x->right;
            if (y->child == x) y->child = x->right;
        }
        y->degree--;
        x->parent = nullptr;
        x->mark = false;
        x->left = min_->left;
        x->right = min_;
        min_->left->right = x;
        min_->left = x;
    }

    // 级联切断：若 y 已被切过一次(mark)，则继续把 y 也切下来
    void cascadingCut(Node* y) {
```

```
    Node* z = y->parent;
    if (z) {
        if (!y->mark) y->mark = true;
        else { cut(y, z); cascadingCut(z); }
    }
}

// 合并根链表：每种度数至多保留一棵根，其余挂为它的孩子
//
// 关于 A_ 与 used_ 的说明：A_[d] 是「当前度数恰为 d 的那棵根」。
// 直接每次 memset(A_) 的代价与 A_ 的大小成正比（D(n) + 1），而每次 extract-min
// 都需要 consolidate，所以这里改用 used_ 记录本回合真正写过的度数，
// 只在最后复位这几项，代价与「根数」成正比。
void consolidate() {
    // 先把当前根链表的结点收集到临时数组（遍历期间会改动链表，故先收集）
    roots_.clear();
    Node* w = min_;
    do { roots_.push_back(w); w = w->right; } while (w != min_);

    used_.clear();
    for (size_t i = 0; i < roots_.size(); ++i) {
        Node* x = roots_[i];
        int d = x->degree;
        while (d < (int)A_.size() && A_[d]) {
            Node* y = A_[d];
            if (less(y->key, x->key)) std::swap(x, y);  // 保证 x 是较小者
            linkFib(y, x);                              // y 成为 x 的孩子
            A_[d] = nullptr;
            ++d;
        }
        // 度数可能超过预设的 A_ 长度（理论上 D(n) = O(log n)，此分支不会触发），
        // 这里做一次兜底扩容，保证任何输入下都不会越界。
        if (d >= (int)A_.size()) A_.resize((size_t)d + 1, nullptr);
        used_.push_back(d);     // 循环退出时 A_[d] 必为 nullptr
        A_[d] = x;
    }

    // 由 A_ 重建根链表（同时求出新的 min_）
    min_ = nullptr;
    for (size_t i = 0; i < used_.size(); ++i) {
        const int d = used_[i];
        Node* nd = A_[d];
        if (!nd) continue;          // 该度数在前面已被消费过
        A_[d] = nullptr;            // 消费掉，避免同一结点被插入两次
        if (!min_) {
            min_ = nd;
            nd->left = nd; nd->right = nd;
        }
        else {
            nd->right = min_->right;
            min_->right->left = nd;
            nd->left = min_;
            min_->right = nd;
            if (less(nd->key, min_->key)) min_ = nd;
        }
    }
    used_.clear();
}

// 把 y 从根链表摘下并作为 x 的孩子（要求 degree(y) == degree(x)）
void linkFib(Node* y, Node* x) {
    // 从根链表移除 y
    y->left->right = y->right;
    y->right->left = y->left;
    // 挂到 x 的孩子链表中
    y->parent = x;
    if (!x->child) {
        x->child = y;
        y->left = y; y->right = y;
```

```
        }
        else {
            y->right = x->child->right;
            x->child->right->left = y;
            y->left = x->child;
            x->child->right = y;
        }
        x->degree++;
        y->mark = false;
    }

    bool less(Key a, Key b) { ++cmps_; return a < b; }

    std::deque<Node>    pool_;     // 预分配节点池；用 deque 保证扩容时已有结点地址不变
    std::vector<Node*>  handle_;   // handle_[v]：顶点 v 对应的堆结点
    std::vector<char>   in_;       // in_[v]：顶点 v 是否在堆中
    std::vector<int>    touched_;  // 本次 query 插入过的顶点
    std::vector<Node*>  A_;        // 度数 -> 根
    std::vector<int>    used_;     // A_ 中本回合被写过的度数（用于 O(根数) 复位）
    std::vector<Node*>  roots_;    // consolidate 用的临时根集合
    Node* min_ = nullptr;
    int   nfree_ = 0;              // 节点池已用数量
    int   n_ = 0;                  // 堆中元素个数
    long long cmps_ = 0;
};

// ============================================================================
//  4) 惰性删除二叉堆（Lazy Binary Heap）—额外的对照实现
//
//  这是工程上最常用的 Dijkstra 写法：堆里只存 (key, v)，不做 decrease-key，
//  松弛成功就直接 push 一个新副本；extractMin 时若发现 key 不是最新值就丢弃。
//  它的堆元素个数会涨到 O(m)，理论复杂度略差，但常数因子小、实现简单。
//  加入它是为了在报告中回答一个关键问题：
//      「既然斐波那契堆理论最优，为什么实践中大家用普通二叉堆?」
// ============================================================================
class LazyBinaryHeap {
public:
    explicit LazyBinaryHeap(int /*n*/) { }

    void clear() { key_.clear(); ver_.clear(); }

    bool empty() const { return key_.empty(); }

    // 插入（或再次插入）顶点 v 及其关键字
    void push(int v, Key k) {
        key_.push_back(k);
        ver_.push_back(v);
        siftUp((int)key_.size() - 1);
    }

    // 弹出最小元素；返回顶点编号，key 通过 outKey 带回
    int extractMin(Key& outKey) {
        const int v = ver_[0];
        outKey = key_[0];
        const Key lk = key_.back();
        const int  lv = ver_.back();
        key_.pop_back();
        ver_.pop_back();
        if (!key_.empty()) {
            key_[0] = lk;
            ver_[0] = lv;
            siftDown(0);
        }
        return v;
    }

    long long cmps() const { return cmps_; }

private:
```

```
    void siftUp(int i) {
        const Key k = key_[i];
        const int v = ver_[i];
        while (i > 0) {
            const int p = (i - 1) >> 1;
            if (!less(k, key_[p])) break;
            key_[i] = key_[p];
            ver_[i] = ver_[p];
            i = p;
        }
        key_[i] = k;
        ver_[i] = v;
    }

    void siftDown(int i) {
        const int n = (int)key_.size();
        const Key k = key_[i];
        const int v = ver_[i];
        for (;;) {
            int c = 2 * i + 1;
            if (c >= n) break;
            if (c + 1 < n && less(key_[c + 1], key_[c])) ++c;
            if (!less(key_[c], k)) break;
            key_[i] = key_[c];
            ver_[i] = ver_[c];
            i = c;
        }
        key_[i] = k;
        ver_[i] = v;
    }

    bool less(Key a, Key b) const { ++cmps_; return a < b; }

    std::vector<Key> key_;
    std::vector<int> ver_;
    mutable long long cmps_ = 0;
};
```

## A.2 src/dijkstra.h（Dijkstra 框架）

```
// ============================================================================
//  dijkstra.h —— 以最小优先队列（模板参数）为核心的 Dijkstra 实现
//
//  提供两个「Runner」类，它们各自持有跨 query 复用的缓冲区：
//      DijkstraRunner<Heap>      —— 可索引堆 + decrease-key 的标准写法
//      LazyDijkstraRunner        —— 二叉堆 + 惰性删除的工程写法（对照用）
//
//  支持两种模式：
//      full = false : 点到点查询。当终点被弹出（即已确定最短距离）时立即停止，
//                     这是 DIMACS P2P 问题的标准做法。
//      full = true  : 完整单源最短路，把源点可达的整个集合都扫描完。
//
//  复杂度（n 结点、m 弧）：
//      二叉堆      : O((n + m) log n)
//      二项堆      : O((n + m) log n)       （但 decrease-key 的常数更大）
//      斐波那契堆  : O(m + n log n)
// ============================================================================
#pragma once

#include <vector>
#include <chrono>
#include "dimacs.h"
#include "heaps.h"

// 一次 Dijkstra 调用的统计信息
struct DijkstraStats {
    long long pops     = 0;   // extractMin 次数
    long long pushes   = 0;   // insert 次数
    long long decs     = 0;   // decreaseKey 次数
    long long arcScans = 0;   // 扫描过的弧数（松弛尝试次数）
    long long cmps     = 0;   // 关键字比较次数
    long long settled  = 0;   // 被确定最短距离的结点数
    // 所有被确定结点的距离之和模 2^62：不同堆之间应逐位一致，用于交叉验证
    unsigned long long sumDist = 0;
};

// ----------------------------------------------------------------------------
//  可索引堆 + decrease-key 版本的 Dijkstra
// ----------------------------------------------------------------------------
template <class Heap>
class DijkstraRunner {
public:
    explicit DijkstraRunner(const Graph& g)
        : g_(g), dist_((size_t)g.n + 1, 0), done_((size_t)g.n + 1, 0),
          heap_(g.n) {
        settled_.reserve((size_t)g.n + 1);
    }

    // 运行一次 Dijkstra：返回 dist[t]；full=true 时 t 无意义，返回值为 0
    Key run(int s, int t, bool full, DijkstraStats& st) {
        const long long c0 = heap_.cmps();      // 本此调用的比较次数增量
        heap_.clear();
        heap_.insert(s, 0);
        dist_[s] = 0;
        Key answer = KEY_INF;                   // 若终点不可达则保持为无穷大

        while (!heap_.empty()) {
            const int u = heap_.extractMin();
            ++st.pops;
            done_[u] = 1;
            settled_.push_back(u);
            ++st.settled;
            st.sumDist += (unsigned long long)dist_[u];
            if (!full && u == t) { answer = dist_[u]; break; }

            const Key du = dist_[u];
            for (int i = g_.head[u]; i < g_.head[u + 1]; ++i) {
```

```
                ++st.arcScans;
                const int v = g_.to[i];
                if (done_[v]) continue;                 // v 已确定，无需松弛
                const Key nd = du + g_.w[i];
                if (heap_.contains(v)) {
                    if (nd < heap_.keyOf(v)) {          // 松弛成功 -> decrease-key
                        heap_.decreaseKey(v, nd);
                        dist_[v] = nd;
                        ++st.decs;
                    }
                }
                else {
                    dist_[v] = nd;                      // 首次发现 -> insert
                    heap_.insert(v, nd);
                    ++st.pushes;
                }
            }
        }

        // 清理 done_ 标记（只清理本次碰过的结点，O(工作量)）
        for (size_t i = 0; i < settled_.size(); ++i) done_[settled_[i]] = 0;
        settled_.clear();
        st.cmps += heap_.cmps() - c0;
        return answer;
    }

private:
    const Graph& g_;
    std::vector<Key> dist_;
    std::vector<char> done_;
    std::vector<int> settled_;
    Heap heap_;
};

// ---------------------------------------------------------------------------
//  惰性删除版本的 Dijkstra（只搭配 LazyBinaryHeap 使用）
//
//  不做 decrease-key：松弛成功就再 push 一份 (key, v)，弹到陈旧副本便丢弃。
//  堆规模最坏可到 O(m)，但常数因子小，是工程上最常见的写法。
// ---------------------------------------------------------------------------
class LazyDijkstraRunner {
public:
    explicit LazyDijkstraRunner(const Graph& g)
        : g_(g), dist_((size_t)g.n + 1, 0), done_((size_t)g.n + 1, 0),
          seen_((size_t)g.n + 1, 0), heap_(g.n) {}

    Key run(int s, int t, bool full, DijkstraStats& st) {
        const long long c0 = heap_.cmps();      // 本次调用的比较次数增量
        heap_.clear();
        heap_.push(s, 0);
        dist_[s] = 0;
        seen_[s] = 1;
        touchedSeen_.push_back(s);
        Key answer = KEY_INF;                   // 若终点不可达则保持为无穷大

        while (!heap_.empty()) {
            Key k = 0;
            const int u = heap_.extractMin(k);
            ++st.pops;
            if (done_[u]) continue;                 // 陈旧副本，直接丢弃
            if (k != dist_[u]) continue;            // 关键字不是最新值
            done_[u] = 1;
            settled_.push_back(u);
            ++st.settled;
            st.sumDist += (unsigned long long)k;
            if (!full && u == t) { answer = k; break; }

            for (int i = g_.head[u]; i < g_.head[u + 1]; ++i) {
                ++st.arcScans;
```

```
                const int v = g_.to[i];
                if (done_[v]) continue;
                const Key nd = k + g_.w[i];
                if (!seen_[v] || nd < dist_[v]) {   // 松弛成功 -> 直接再 push 一份
                    dist_[v] = nd;
                    if (!seen_[v]) { seen_[v] = 1; touchedSeen_.push_back(v); }
                    heap_.push(v, nd);
                    ++st.pushes;
                }
            }
        }

        for (size_t i = 0; i < settled_.size(); ++i) done_[settled_[i]] = 0;
        settled_.clear();
        for (size_t i = 0; i < touchedSeen_.size(); ++i) seen_[touchedSeen_[i]] = 0;
        touchedSeen_.clear();
        st.cmps += heap_.cmps() - c0;
        return answer;
    }

private:
    const Graph& g_;
    std::vector<Key> dist_;
    std::vector<char> done_;
    std::vector<char> seen_;
    std::vector<int> settled_;
    std::vector<int> touchedSeen_;
    LazyBinaryHeap heap_;
};
```

## A.3 src/dimacs.h（图与查询的读取）

```
// ============================================================================
//  dimacs.h —— DIMACS Challenge 9 图数据 / 查询数据的读取
//
//  文件格式（来自 Challenge 9 官方说明）：
//    .gr  文件：  c 注释行
//                 p sp <n> <m>        结点数、弧数
//                 a <u> <v> <w>       一条从 u 到 v、权为 w 的弧
//    .p2p 文件：  c 注释行
//                 p aux sp p2p <k>    k 组查询
//                 q <s> <t>           源点 s、终点 t
//    .ss  文件：  p aux sp ss <k> / s <s>   单源查询
//
//  实现要点：
//   1）一次性把整个文件读进内存，再用自写的手工扫描器解析，比 >> 快得多；
//      解析分两遍：第一遍只统计每个结点的出度，第二遍再填写 CSR 数组，
//      因此除了图本身之外不需要 (u,v,w) 三元组的临时存储。
//   2）图用 CSR（压缩稀疏行）存储：head[] + to[] + w[]，
//      遍历某结点所有出边是连续内存访问，缓存友好，这对 Dijkstra 的实测很关键。
// ============================================================================
#pragma once

#include <vector>
#include <string>
#include <cstdio>
#include <cstdlib>
#include <cstring>

// ----------------------------------------------------------------------------
//  图：CSR 结构
// ----------------------------------------------------------------------------
struct Graph {
    int n = 0;                      // 结点数，编号 1..n
    std::vector<int> head;          // 长度 n+2：head[u]..head[u+1]-1 是 u 的出边区间
    std::vector<int> to;            // 弧头
    std::vector<int> w;             // 弧权

    long long nArcs() const { return (long long)to.size(); }

    // 从边表构造 CSR（测试程序用）
    static Graph fromEdges(int n, const std::vector<int>& eu,
                           const std::vector<int>& ev,
                           const std::vector<int>& ew);
};

inline Graph Graph::fromEdges(int n, const std::vector<int>& eu,
                              const std::vector<int>& ev,
                              const std::vector<int>& ew) {
    Graph g;
    g.n = n;
    g.head.assign((size_t)n + 2, 0);
    const size_t m = eu.size();
    for (size_t i = 0; i < m; ++i) ++g.head[(size_t)eu[i] + 1];
    for (int u = 1; u <= n; ++u) g.head[u + 1] += g.head[u];
    g.to.resize(m);
    g.w.resize(m);
    std::vector<int> cursor(g.head.begin(), g.head.end());
    for (size_t i = 0; i < m; ++i) {
        const int u = eu[i];
        const int p = cursor[u]++;
        g.to[p] = ev[i];
        g.w[p] = ew[i];
    }
    return g;
}

// ----------------------------------------------------------------------------
//  文件读取工具
```

```
// ---------------------------------------------------------------------------

// 把整个文件读入内存；失败则返回空
inline std::string readWholeFile(const std::string& path) {
    std::FILE* f = std::fopen(path.c_str(), "rb");
    if (!f) return std::string();
    std::fseek(f, 0, SEEK_END);
    const long sz = std::ftell(f);
    std::fseek(f, 0, SEEK_SET);
    std::string buf;
    if (sz > 0) {
        buf.resize((size_t)sz);
        const size_t got = std::fread(&buf[0], 1, (size_t)sz, f);
        buf.resize(got);
    }
    std::fclose(f);
    return buf;
}

// 手工整数扫描：跳过非数字字符，读一个整数
inline long long scanInt(const char*& p, const char* end) {
    while (p < end && (*p < '0' || *p > '9') && *p != '-') ++p;
    bool neg = false;
    if (p < end && *p == '-') { neg = true; ++p; }
    long long x = 0;
    while (p < end && *p >= '0' && *p <= '9') { x = x * 10 + (*p - '0'); ++p; }
    return neg ? -x : x;
}

// 跳过当前行剩余部分，返回下一行的起点（若已到文件末尾返回 end）
inline const char* nextLine(const char* p, const char* end) {
    while (p < end && *p != '\n') ++p;
    if (p < end) ++p;
    return p;
}

// ---------------------------------------------------------------------------
//  读取 .gr 图文件
// ---------------------------------------------------------------------------
inline Graph loadGraph(const std::string& path, long long* outLoadMs = nullptr) {
    const std::string buf = readWholeFile(path);
    if (buf.empty()) {
        std::fprintf(stderr, "[dimacs] cannot read graph file: %s\n", path.c_str());
        std::exit(1);
    }
    Graph g;
    const char* p = buf.data();
    const char* end = p + buf.size();

    // ---- 第 1 遍：找问题行拿到 n，并统计每个结点的出度 ----
    std::vector<int> deg;
    for (const char* q = p; q < end; ) {
        const char c = *q;
        if (c == 'p') {
            // p sp n m
            const char* r = q + 1;
            g.n = (int)scanInt(r, end);          // 跳过 "sp" 后读到 n
            scanInt(r, end);                     // m（由 to.size() 实际决定）
            deg.assign((size_t)g.n + 2, 0);
            q = nextLine(q, end);
        }
        else if (c == 'a') {
            const char* r = q + 1;
            const int u = (int)scanInt(r, end);
            scanInt(r, end);                     // v，本条不需要
            (void)scanInt(r, end);               // w
            ++deg[(size_t)u + 1];                // 计入 u 的出度
            q = nextLine(q, end);
        }
```

```
        else {
            q = nextLine(q, end);
        }
    }
    if (g.n <= 0) {
        std::fprintf(stderr, "[dimacs] bad graph header: %s\n", path.c_str());
        std::exit(1);
    }

    // ---- 前缀和得到 head[] ----
    g.head.assign((size_t)g.n + 2, 0);
    for (int u = 1; u <= g.n; ++u) g.head[u + 1] = g.head[u] + deg[u + 1];
    const long long m = g.head[g.n + 1];
    g.to.resize((size_t)m);
    g.w.resize((size_t)m);
    std::vector<int> cursor(g.head.begin(), g.head.end());

    // ---- 第 2 遍: 填 CSR ----
    for (const char* q = p; q < end; ) {
        if (*q == 'a') {
            const char* r = q + 1;
            const int u = (int)scanInt(r, end);
            const int v = (int)scanInt(r, end);
            const int w = (int)scanInt(r, end);
            const int pos = cursor[u]++;
            g.to[pos] = v;
            g.w[pos] = w;
        }
        q = nextLine(q, end);
    }
    (void)outLoadMs;
    return g;
}

// ---------------------------------------------------------------------------
//  读取查询文件 (.p2p 或 .ss)
// ---------------------------------------------------------------------------
inline bool loadQueries(const std::string& path, std::vector<int>& srcs,
                        std::vector<int>& dsts) {
    const std::string buf = readWholeFile(path);
    if (buf.empty()) return false;
    const char* p = buf.data();
    const char* end = p + buf.size();
    for (const char* q = p; q < end; ) {
        const char c = *q;
        if (c == 'q') {                         // q s t
            const char* r = q + 1;
            const int s = (int)scanInt(r, end);
            const int t = (int)scanInt(r, end);
            srcs.push_back(s);
            dsts.push_back(t);
        }
        else if (c == 's') {                    // s S (单源)
            const char* r = q + 1;
            const int s = (int)scanInt(r, end);
            srcs.push_back(s);
            dsts.push_back(-1);
        }
        q = nextLine(q, end);
    }
    return !srcs.empty();
}
```

## A.4 src/bench.cpp（性能测试程序）

```cpp
// ============================================================================
//  bench.cpp — 性能测试主程序
//
//  用法:
//    bench --graph <file.gr> [--net NAME] [--p2p <file.p2p>|--ss <file.ss>]
//          [--heaps bin,binom,fib,lazy] [--full-sources K] [--seed S]
//          [--out results.csv] [--dump out.txt]
//
//  程序流程:
//    1) 读入 CSR 图（加载时间单独打印，不计入算法计时）;
//    2) 读入查询集合（.p2p 的点对 或 .ss 的单源）;
//    3) 对每一种堆：跑完整个查询集合，统计总时间 / 平均时间 / 校验和 / 堆操作次数;
//    4) 可选：对 K 个随机源点跑「完整单源最短路」(不提前终止)，
//       用于观察「必须扫描整个连通分量」时各种堆的差异;
//    5) 结果以 CSV 形式输出（stdout，同时可写入 --out 文件）。
//
//  校验和（checksum）取「所有被确定结点的距离之和」模 2^62，与 DIMACS 的一致性
//  检查思路一致，用来验证不同堆得到的最短路长度完全一致。
// ============================================================================
#include <cstdio>
#include <cstring>
#include <cstdlib>
#include <string>
#include <vector>
#include <random>
#include <chrono>

#include "dimacs.h"
#include "heaps.h"
#include "dijkstra.h"

static double nowMs() {
    using namespace std::chrono;
    return duration<double, std::milli>(steady_clock::now().time_since_epoch()).count();
}

struct Args {
    std::string graph, net = "?", p2p, ss, heaps = "bin,binom,fib,lazy", out, dump;
    int fullSources = 0;
    unsigned seed = 20241024u;
};

static Args parseArgs(int argc, char** argv) {
    Args a;
    for (int i = 1; i < argc; ++i) {
        const std::string k = argv[i];
        auto next = [&]() -> std::string { return (i + 1 < argc) ? argv[++i] : std::string(); };
        if (k == "--graph")             a.graph = next();
        else if (k == "--net")          a.net = next();
        else if (k == "--p2p")          a.p2p = next();
        else if (k == "--ss")           a.ss = next();
        else if (k == "--heaps")        a.heaps = next();
        else if (k == "--out")          a.out = next();
        else if (k == "--dump")         a.dump = next();
        else if (k == "--full-sources") a.fullSources = std::atoi(next().c_str());
        else if (k == "--seed")         a.seed = (unsigned)std::strtoul(next().c_str(), nullptr, 10);
        else { std::fprintf(stderr, "[bench] unknown option %s\n", k.c_str()); std::exit(2); }
    }
    if (a.graph.empty()) { std::fprintf(stderr, "[bench] --graph required\n"); std::exit(2); }
    return a;
}

static const char* CSV_HEADER =
    "net,nodes,arcs,heap,mode,runs,total_ms,avg_ms,checksum,"
    "pops,pushes,decrease_keys,arc_scans,cmps,settled,unreachable";

// 判断 --heaps 中是否精确包含某个名字（避免 "bin" 匹配到 "binom"）
static bool hasHeap(const std::string& list, const char* name) {
```

```
    size_t p = 0;
    const std::string nm(name);
    while (p <= list.size()) {
        const size_t q = list.find(',', p);
        const std::string tok = list.substr(p, (q == std::string::npos ? list.size() : q) - p);
        if (tok == nm) return true;
        if (q == std::string::npos) break;
        p = q + 1;
    }
    return false;
}

// ---------------------------------------------------------------------------
//  点对查询实验：跑完整个查询集合（提前终止）
//
//  dumpPrefix 非空时，把每次查询的明细写成 CSV：
//      <prefix>.<heapName>.csv  内容为 idx,src,dst,dist,settled_ms,settled,pops
//  便于后续按「搜索规模」分桶分析（报告中 runtime vs. 搜索规模的图）。
// ---------------------------------------------------------------------------
template <class Runner>
static void benchP2P(const char* heapName, const Graph& g, Runner& R,
                     const std::vector<int>& srcs, const std::vector<int>& dsts,
                     const Args& a, std::FILE* out, const std::string& dumpPrefix) {
    DijkstraStats st;
    long long unreachable = 0;

    std::FILE* dump = nullptr;
    if (!dumpPrefix.empty()) {
        const std::string fn = dumpPrefix + "." + heapName + ".csv";
        dump = std::fopen(fn.c_str(), "w");
        if (dump) std::fprintf(dump, "idx,src,dst,dist,ms,settled,pops\n");
    }

    const double t0 = nowMs();
    for (size_t i = 0; i < srcs.size(); ++i) {
        DijkstraStats one;
        const double q0 = nowMs();
        const Key d = R.run(srcs[i], dsts[i], false, one);
        const double q1 = nowMs();
        if (d >= KEY_INF / 2) ++unreachable;
        if (dump) {
            std::fprintf(dump, "%zu,%d,%d,%lld,%.6f,%lld,%lld\n",
                         i, srcs[i], dsts[i], (long long)(d >= KEY_INF / 2 ? -1 : d),
                         q1 - q0, one.settled, one.pops);
        }
        st.pops += one.pops; st.pushes += one.pushes; st.decs += one.decs;
        st.arcScans += one.arcScans; st.settled += one.settled; st.sumDist += one.sumDist;
        st.cmps += one.cmps;
    }
    const double t1 = nowMs();
    if (dump) std::fclose(dump);

    const double total = t1 - t0;
    const double avg = total / (double)srcs.size();
    const unsigned long long checksum = st.sumDist & ((1ULL << 62) - 1);

    std::fprintf(out, "%s,%d,%lld,%s,p2p,%zu,%.3f,%.6f,%llu,%lld,%lld,%lld,%lld,%lld,%lld,%lld\n",
                 a.net.c_str(), g.n, g.nArcs(), heapName, srcs.size(), total, avg, checksum,
                 st.pops, st.pushes, st.decs, st.arcScans, st.cmps, st.settled, unreachable);
    std::fflush(out);
    std::fprintf(stderr, "  [p2p ] %-11s runs=%zu total=%9.2fms avg=%8.5fms "
                         "settled/q=%lld pops=%lld decs=%lld arcs=%lld\n",
                 heapName, srcs.size(), total, avg, st.settled / (long long)srcs.size(),
                 st.pops, st.decs, st.arcScans);
}

// ---------------------------------------------------------------------------
//  完整单源实验：K 个随机源点，不提前终止
// ---------------------------------------------------------------------------
template <class Runner>
```

```
static void benchFull(const char* heapName, const Graph& g, Runner& R,
                      const std::vector<int>& sources, const Args& a, std::FILE* out) {
    DijkstraStats st;

    const double t0 = nowMs();
    for (size_t i = 0; i < sources.size(); ++i) R.run(sources[i], -1, true, st);
    const double t1 = nowMs();

    const double total = t1 - t0;
    const double avg = total / (double)sources.size();
    const unsigned long long checksum = st.sumDist & ((1ULL << 62) - 1);

    std::fprintf(out, "%s,%d,%lld,%s,full,%zu,%.3f,%.6f,%llu,%lld,%lld,%lld,%lld,%lld,%lld,%lld\n",
                 a.net.c_str(), g.n, g.nArcs(), heapName, sources.size(), total, avg, checksum,
                 st.pops, st.pushes, st.decs, st.arcScans, st.cmps, st.settled, 0LL);
    std::fflush(out);
    std::fprintf(stderr, "  [full] %-11s runs=%zu total=%9.2fms avg=%8.5fms "
                         "settled/run=%lld pops=%lld decs=%lld cmps=%lld\n",
                 heapName, sources.size(), total, avg, st.settled / (long long)sources.size(),
                 st.pops, st.decs, st.cmps);
}

int main(int argc, char** argv) {
    const Args a = parseArgs(argc, argv);

    std::fprintf(stderr, "[bench] loading graph %s ...\n", a.graph.c_str());
    const double t0 = nowMs();
    const Graph g = loadGraph(a.graph);
    const double t1 = nowMs();
    std::fprintf(stderr, "[bench] graph loaded: n=%d m=%lld in %.0f ms\n",
                 g.n, g.nArcs(), t1 - t0);

    std::vector<int> srcs, dsts;
    const std::string qfile = a.p2p.empty() ? a.ss : a.p2p;
    if (!qfile.empty()) {
        if (!loadQueries(qfile, srcs, dsts)) {
            std::fprintf(stderr, "[bench] cannot read query file: %s\n", qfile.c_str());
            std::exit(1);
        }
        std::fprintf(stderr, "[bench] loaded %zu queries from %s\n", srcs.size(), qfile.c_str());
    }

    std::vector<int> fullSources;
    if (a.fullSources > 0) {
        std::mt19937 rng(a.seed ^ 0x9e3779b9u);
        std::uniform_int_distribution<int> U(1, g.n);
        for (int i = 0; i < a.fullSources; ++i) fullSources.push_back(U(rng));
    }

    std::FILE* out = stdout;
    if (!a.out.empty()) {
        bool needHeader = true;
        if (std::FILE* probe = std::fopen(a.out.c_str(), "rb")) {   // 已有内容则不再写表头
            std::fseek(probe, 0, SEEK_END);
            needHeader = std::ftell(probe) == 0;
            std::fclose(probe);
        }
        out = std::fopen(a.out.c_str(), "a");
        if (!out) { std::fprintf(stderr, "[bench] cannot open --out\n"); std::exit(1); }
        if (needHeader) std::fprintf(out, "%s\n", CSV_HEADER);
    }
    std::FILE* dumpFile = nullptr;
    (void)dumpFile;

    if (hasHeap(a.heaps, "bin")) {
        DijkstraRunner<BinaryHeap> R(g);
        if (!srcs.empty()) benchP2P("binary", g, R, srcs, dsts, a, out, a.dump);
        if (!fullSources.empty()) benchFull("binary", g, R, fullSources, a, out);
    }
    if (hasHeap(a.heaps, "binom")) {
```

```
        DijkstraRunner<BinomialHeap> R(g);
        if (!srcs.empty()) benchP2P("binomial", g, R, srcs, dsts, a, out, a.dump);
        if (!fullSources.empty()) benchFull("binomial", g, R, fullSources, a, out);
    }
    if (hasHeap(a.heaps, "fib")) {
        DijkstraRunner<FibonacciHeap> R(g);
        if (!srcs.empty()) benchP2P("fibonacci", g, R, srcs, dsts, a, out, a.dump);
        if (!fullSources.empty()) benchFull("fibonacci", g, R, fullSources, a, out);
    }
    if (hasHeap(a.heaps, "lazy")) {
        LazyDijkstraRunner R(g);
        if (!srcs.empty()) benchP2P("binary-lazy", g, R, srcs, dsts, a, out, a.dump);
        if (!fullSources.empty()) benchFull("binary-lazy", g, R, fullSources, a, out);
    }

    if (out != stdout) std::fclose(out);
    return 0;
}
```

## A.5 src/selftest.cpp（正确性测试）

```cpp
// ============================================================================
//  selftest.cpp —— 正确性测试（回归测试 + 随机对拍）
//
//  分三层：
//   (A) 堆操作的确定性用例：一批已知的 insert / decreaseKey / extractMin 序列，
//       手工核对弹出顺序。
//   (B) 堆操作的随机对拍：随机序列与「每次线性扫描找最小」的朴素参考实现逐条比对，
//       覆盖三种堆 + 惰性二叉堆。
//   (C) Dijkstra 对拍：在随机有向图上，把 4 种实现算出的最短路与 SPFA
//       （队列式 Bellman-Ford，独立实现、不依赖任何堆）逐点比对；
//       同时对同一批查询校验各种堆的 checksum 完全一致。
//
//  任意一项失败都会打印出错细节并以非零状态退出，方便脚本化回归。
// ============================================================================
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <random>
#include <string>
#include <algorithm>

#include "dimacs.h"
#include "heaps.h"
#include "dijkstra.h"

static int g_failures = 0;

static void check(bool ok, const std::string& what) {
    if (!ok) {
        std::printf("  [FAIL] %s\n", what.c_str());
        ++g_failures;
    }
}

// ============================================================================
//  (A) 堆的确定性用例
// ============================================================================
template <class Heap>
static void scriptedCase(const char* name) {
    const int n = 8;
    Heap h(n);
    h.clear();
    // 依次插入顶点 1..8，关键字故意乱序
    const long long keys[9] = {0, 50, 20, 70, 10, 60, 30, 80, 40};
    for (int v = 1; v <= n; ++v) h.insert(v, keys[v]);
    // 把顶点 7 的关键字从 80 降到 5
    h.decreaseKey(7, 5);
    // 把顶点 2 的关键字从 20 降到 1
    h.decreaseKey(2, 1);

    // 期望弹出顺序按关键字升序：
    //   更新后关键字 v2=1, v7=5, v4=10, v6=30, v8=40, v1=50, v5=60, v3=70
    const int expect[8] = {2, 7, 4, 6, 8, 1, 5, 3};
    for (int i = 0; i < n; ++i) {
        const int got = h.extractMin();
        check(got == expect[i], std::string(name) + " scripted: extract #" + std::to_string(i) +
                                " got " + std::to_string(got) + " expect " + std::to_string(expect[i]));
    }
    check(h.empty(), std::string(name) + " scripted: should be empty at the end");
}

// ============================================================================
//  (B) 堆操作随机对拍
// ============================================================================
// 朴素参考实现：O(n) 线性扫描找最小
struct RefHeap {
    std::vector<Key> key;
```

```
    std::vector<char> in;
    int sz = 0;
    explicit RefHeap(int n) : key((size_t)n + 1, 0), in((size_t)n + 1, 0) {}
    void clear() { std::fill(in.begin(), in.end(), 0); sz = 0; }
    int  size() const { return sz; }
    bool contains(int v) const { return in[v] != 0; }
    Key  keyOf(int v) const { return key[v]; }
    void insert(int v, Key k) { key[v] = k; in[v] = 1; ++sz; }
    void decreaseKey(int v, Key k) { key[v] = k; }
    int extractMin() {
        int best = -1;
        for (size_t v = 1; v < in.size(); ++v)
            if (in[v] && (best < 0 || key[v] < key[best])) best = (int)v;
        if (best < 0) return -1;
        in[best] = 0;
        --sz;
        return best;
    }
};

template <class Heap>
static void stressHeap(const char* name, unsigned seed) {
    const int n = 2000;
    const int OPS = 200000;
    Heap h(n);
    RefHeap ref(n);
    h.clear();
    ref.clear();

    // 关键字编码为 key = value * n + v (value 随机、v 为顶点编号)。
    // 这样所有关键字互不相同，最小值唯一，从而可以严格比较「弹出了哪个顶点」，
    // 不会因为「并列最小」时两种实现任意选择不同的顶点而误报。
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> Uv(1, n);
    std::uniform_int_distribution<int> Uw(1, 1000000);
    int mismatch = 0;

    for (int op = 0; op < OPS && mismatch == 0; ++op) {
        const int r = (int)(rng() % 100);
        if (r < 55) {                       // 55%: insert
            const int v = Uv(rng);
            if (ref.contains(v)) {          // 已在堆中则退化为 decreaseKey
                const Key nk = std::min(ref.keyOf(v), (Key)Uw(rng) * n + v);
                h.decreaseKey(v, nk); ref.decreaseKey(v, nk);
            } else {
                const Key k = (Key)Uw(rng) * n + v;
                h.insert(v, k); ref.insert(v, k);
            }
        } else if (r < 85 && ref.size() > 0) {    // 30%: decreaseKey (只对已存在的顶点)
            int v = -1;
            for (int tries = 0; tries < 50; ++tries) {
                const int c = Uv(rng);
                if (ref.contains(c)) { v = c; break; }
            }
            if (v > 0) {
                // 保持 v 的后缀不变，只把 value 减小
                const Key nk = ref.keyOf(v) - (Key)(rng() % 100 + 1) * n;
                h.decreaseKey(v, nk); ref.decreaseKey(v, nk);
            }
        } else if (ref.size() > 0) {        // 其余: extractMin (比较返回值与关键字)
            const int a = h.extractMin();
            const int b = ref.extractMin();
            if (a != b) ++mismatch;
        }
    }
    check(mismatch == 0, std::string(name) + " stress: extractMin mismatch");

    // 收尾：把剩下的全部弹出，核对顺序
    for (;;) {
```

```
        // 参考实现先取最小，再让被测堆取
        if (ref.size() == 0) break;
        const int b = ref.extractMin();
        const int a = h.extractMin();
        if (a != b) { ++mismatch; break; }
    }
    check(mismatch == 0, std::string(name) + " stress: drain mismatch");
    check(h.empty(), std::string(name) + " stress: heap should be empty after drain");
    std::printf("  [ok ] %-14s stress ops=%d comparisons=%lld\n", name, OPS, h.cmps());
}

// 惰性堆单独对拍（语义不同：允许重复存在）
static void stressLazy(unsigned seed) {
    const int n = 2000;
    const int OPS = 200000;
    LazyBinaryHeap h(n);
    std::vector<std::pair<Key, int> > ref;      // 参考：直接存所有 (key,v) 副本
    std::mt19937 rng(seed);
    std::uniform_int_distribution<int> Uv(1, n);
    std::uniform_int_distribution<int> Uw(1, 1000000);
    int mismatch = 0;
    for (int op = 0; op < OPS && !mismatch; ++op) {
        if (rng() % 100 < 60) {
            const int v = Uv(rng);
            const Key k = (Key)Uw(rng) * n + v;      // 关键字唯一，避免并列最小时的选择歧义
            h.push(v, k);
            ref.push_back(std::make_pair(k, v));
        } else if (!ref.empty()) {
            std::size_t best = 0;
            for (std::size_t i = 1; i < ref.size(); ++i) if (ref[i] < ref[best]) best = i;
            const Key rk = ref[best].first;
            const int  rv = ref[best].second;
            ref[best] = ref.back();
            ref.pop_back();
            Key k;
            const int v = h.extractMin(k);
            if (k != rk || v != rv) ++mismatch;
        }
    }
    check(mismatch == 0, "LazyBinaryHeap stress: mismatch");
    std::printf("  [ok ] %-14s stress ops=%d\n", "lazy-binary", OPS);
}

// ============================================================================
//  (C) Dijkstra 对拍
// ============================================================================
// SPFA / 队列式 Bellman-Ford：独立参考实现，完全不使用优先队列
static std::vector<Key> spfa(const Graph& g, int s) {
    std::vector<Key> d((size_t)g.n + 1, KEY_INF);
    std::vector<char> inq((size_t)g.n + 1, 0);
    std::vector<int> q((size_t)g.n + 1);
    int qh = 0, qt = 0, qn = 0;
    d[s] = 0;
    q[qt] = s; qt = (qt + 1) % (g.n + 1); ++qn; inq[s] = 1;
    while (qn) {
        const int u = q[qh]; qh = (qh + 1) % (g.n + 1); --qn; inq[u] = 0;
        for (int i = g.head[u]; i < g.head[u + 1]; ++i) {
            const int v = g.to[i];
            const Key nd = d[u] + g.w[i];
            if (nd < d[v]) {
                d[v] = nd;
                if (!inq[v]) { q[qt] = v; qt = (qt + 1) % (g.n + 1); ++qn; inq[v] = 1; }
            }
        }
    }
    return d;
}

template <class Runner>
static void crossCheck(const char* name, const Graph& g, Runner& R,
```

```
                    const std::vector<int>& srcs, const std::vector<int>& dsts,
                    unsigned long long& checksumOut) {
    DijkstraStats st;
    unsigned long long cs = 0;
    for (size_t i = 0; i < srcs.size(); ++i) {
        const Key d = R.run(srcs[i], dsts[i], false, st);
        cs += (unsigned long long)d;
    }
    checksumOut = cs & ((1ULL << 62) - 1);
    (void)name;
}

static void dijkstraCrossTest(unsigned seed) {
    // 随机有向图
    std::mt19937 rng(seed);
    const int n = 300;
    const int m = 4000;
    std::uniform_int_distribution<int> Uv(1, n);
    std::uniform_int_distribution<int> Uw(1, 500);
    std::vector<int> eu, ev, ew;
    for (int i = 0; i < m; ++i) {
        const int u = Uv(rng), v = Uv(rng);
        if (u == v) continue;
        eu.push_back(u); ev.push_back(v); ew.push_back(Uw(rng));
    }
    const Graph g = Graph::fromEdges(n, eu, ev, ew);

    // 60 组随机查询
    const int Q = 60;
    std::vector<int> srcs, dsts;
    for (int i = 0; i < Q; ++i) { srcs.push_back(Uv(rng)); dsts.push_back(Uv(rng)); }

    // 参考解：直接对每个源点跑 SPFA
    unsigned long long refSum = 0;
    for (int i = 0; i < Q; ++i) {
        const std::vector<Key> d = spfa(g, srcs[i]);
        refSum += (unsigned long long)d[dsts[i]];
    }
    const unsigned long long refCs = refSum & ((1ULL << 62) - 1);

    // 各种实现
    unsigned long long csBin = 0, csBinom = 0, csFib = 0, csLazy = 0;
    {
        DijkstraRunner<BinaryHeap>   R(g); crossCheck("binary",   g, R, srcs, dsts, csBin);
    }
    {
        DijkstraRunner<BinomialHeap> R(g); crossCheck("binomial", g, R, srcs, dsts, csBinom);
    }
    {
        DijkstraRunner<FibonacciHeap> R(g); crossCheck("fibonacci", g, R, srcs, dsts, csFib);
    }
    {
        LazyDijkstraRunner R(g); crossCheck("lazy", g, R, srcs, dsts, csLazy);
    }

    std::printf("  [ref] SPFA checksum      = %llu\n", refCs);
    std::printf("  [ok ] binary checksum    = %llu\n", csBin);
    std::printf("  [ok ] binomial checksum  = %llu\n", csBinom);
    std::printf("  [ok ] fibonacci checksum = %llu\n", csFib);
    std::printf("  [ok ] lazy checksum      = %llu\n", csLazy);
    check(csBin == refCs,    "dijkstra: binary   != SPFA reference");
    check(csBinom == refCs,  "dijkstra: binomial != SPFA reference");
    check(csFib == refCs,    "dijkstra: fibonacci!= SPFA reference");
    check(csLazy == refCs,   "dijkstra: lazy     != SPFA reference");

    // 完整单源（不提前终止）也要一致
    unsigned long long fBin = 0, fBinom = 0, fFib = 0, fRef = 0;
    {
        DijkstraRunner<BinaryHeap> R(g); DijkstraStats st;
        for (int i = 0; i < 10; ++i) { R.run(srcs[i], -1, true, st); }
```

```
        fBin = st.sumDist & ((1ULL << 62) - 1);
    }
    {
        DijkstraRunner<BinomialHeap> R(g); DijkstraStats st;
        for (int i = 0; i < 10; ++i) { R.run(srcs[i], -1, true, st); }
        fBinom = st.sumDist & ((1ULL << 62) - 1);
    }
    {
        DijkstraRunner<FibonacciHeap> R(g); DijkstraStats st;
        for (int i = 0; i < 10; ++i) { R.run(srcs[i], -1, true, st); }
        fFib = st.sumDist & ((1ULL << 62) - 1);
    }
    for (int i = 0; i < 10; ++i) {
        const std::vector<Key> d = spfa(g, srcs[i]);
        for (int v = 1; v <= n; ++v) if (d[v] < KEY_INF) fRef += (unsigned long long)d[v];
    }
    fRef &= ((1ULL << 62) - 1);
    std::printf("  [full] checksums bin=%llu binom=%llu fib=%llu ref=%llu\n",
                fBin, fBinom, fFib, fRef);
    check(fBin == fRef && fBinom == fRef && fFib == fRef,
          "dijkstra: full SSSP checksum mismatch");
}

// ============================================================================
//  专用小用例：手工可验证的最短路
// ============================================================================
static void handMadeTest() {
    //  CLRS 图 24.6 的简化版：
    //      1 --10--> 2 --1--> 3
    //      1 --5 ---> 4 --4--> 3
    //  从 1 出发：d(2)=10, d(3)=9, d(4)=5
    const int n = 4;
    std::vector<int> eu = {1, 1, 2, 4};
    std::vector<int> ev = {2, 4, 3, 3};
    std::vector<int> ew = {10, 5, 1, 4};
    const Graph g = Graph::fromEdges(n, eu, ev, ew);

    const int expect[5] = {0, 0, 10, 9, 5};
    {
        DijkstraRunner<BinaryHeap> R(g); DijkstraStats st;
        for (int v = 1; v <= n; ++v) {
            const Key d = R.run(1, v, false, st);
            check(d == expect[v], "hand-made: binary d(" + std::to_string(v) + ")=" +
                                  std::to_string(d) + " expect " + std::to_string(expect[v]));
        }
    }
    {
        DijkstraRunner<BinomialHeap> R(g); DijkstraStats st;
        for (int v = 1; v <= n; ++v) {
            const Key d = R.run(1, v, false, st);
            check(d == expect[v], "hand-made: binomial d(" + std::to_string(v) + ")");
        }
    }
    {
        DijkstraRunner<FibonacciHeap> R(g); DijkstraStats st;
        for (int v = 1; v <= n; ++v) {
            const Key d = R.run(1, v, false, st);
            check(d == expect[v], "hand-made: fibonacci d(" + std::to_string(v) + ")");
        }
    }
    {
        LazyDijkstraRunner R(g); DijkstraStats st;
        for (int v = 1; v <= n; ++v) {
            const Key d = R.run(1, v, false, st);
            check(d == expect[v], "hand-made: lazy d(" + std::to_string(v) + ")");
        }
    }
    std::printf("  [ok ] hand-made CLRS-style graph distances {10,9,5}\n");
}
```

```cpp
int main() {
    setvbuf(stdout, nullptr, _IONBF, 0);      // 关闭缓冲，崩溃时也能看到已打印的进度
    std::printf("=============== unit tests ===============\n");
    std::printf("(A) scripted heap cases\n");
    scriptedCase<BinaryHeap>("binary");
    scriptedCase<BinomialHeap>("binomial");
    scriptedCase<FibonacciHeap>("fibonacci");
    std::printf("(B) randomized heap stress (vs. linear-scan reference)\n");
    stressHeap<BinaryHeap>("binary", 1u);
    stressHeap<BinomialHeap>("binomial", 2u);
    stressHeap<FibonacciHeap>("fibonacci", 3u);
    stressLazy(4u);
    std::printf("(C) Dijkstra cross-check (vs. SPFA reference)\n");
    handMadeTest();
    dijkstraCrossTest(5u);

    std::printf("==========================================\n");
    if (g_failures == 0) std::printf("ALL TESTS PASSED\n");
    else                 std::printf("FAILED: %d check(s)\n", g_failures);
    return g_failures == 0 ? 0 : 1;
}
```

# Appendix B　正确性测试完整输出

## B.1　单元测试与随机对拍（`build/selftest.exe`）

```
=============== unit tests ===============
(A) scripted heap cases
(B) randomized heap stress (vs. linear-scan reference)
  [ok ] binary         stress ops=200000 comparisons=1003882
  [ok ] binomial       stress ops=200000 comparisons=632602
  [ok ] fibonacci      stress ops=200000 comparisons=603009
  [ok ] lazy-binary    stress ops=200000
(C) Dijkstra cross-check (vs. SPFA reference)
  [ok ] hand-made CLRS-style graph distances {10,9,5}
  [ref] SPFA checksum      = 14248
  [ok ] binary checksum    = 14248
  [ok ] binomial checksum  = 14248
  [ok ] fibonacci checksum = 14248
  [ok ] lazy checksum      = 14248
  [full] checksums bin=742942 binom=742942 fib=742942 ref=742942
==========================================
ALL TESTS PASSED
```

## B.2　逐查询答案交叉验证（`tools/verify_answers.py`）

下面是对全部数据集、全部实现的逐查询比对结果：同一组 $(s,t)$ 下四种实现返回的 $\mathrm{dist}(t)$ 必须完全相同。`full` 模式下的整体距离校验和也一并逐位核对。

```
BAY: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

CAL: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

COL: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

FLA: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

NE: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

NW: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
  binary-lazy  vs binary       : IDENTICAL

NY: per-query dist[t] cross-check over 4 implementations
  binary       queries=1000, unreachable=0
  binomial     vs binary       : IDENTICAL
  fibonacci    vs binary       : IDENTICAL
```

```
  binary-lazy  vs binary       : IDENTICAL

RESULT: ALL HEAPS AGREE on every query
```

# 作者分工与声明

## 作者分工

| 姓名 / 学号 | 负责内容 | 贡献占比 |
|---|---|---|
| 【】 | 斐波那契堆 / 二项堆实现，摊还分析证明，报告第 2 章 | 【】% |
| 【】 | 二叉堆与惰性删除实现，CSR 与解析器，测试程序 | 【】% |
| 【】 | 实验设计与执行、绘图、结果分析，报告第 3、4 章 | 【】% |

## 声明

We hereby declare that all the work done in this project titled “Shortest Path Algorithm with Heaps” is of our independent effort as a group.

签名：

| | | |
|---|---|---|
| 第 1 作者 | 第 2 作者 | 第 3 作者 |